Nivelul retea

Cuprins

- de ce pachete ?
- organizarea internă
- protocolul IP – adresare si retransmiterea pachetelor
- algoritmi de dirijare
- protocoale de control
- structura internetului
  - protocolul OSPF – Open Shortest Path First
  - protocolul BGP – Border Gateway Protocol
- dirijarea in retele ad hoc
- IPv6

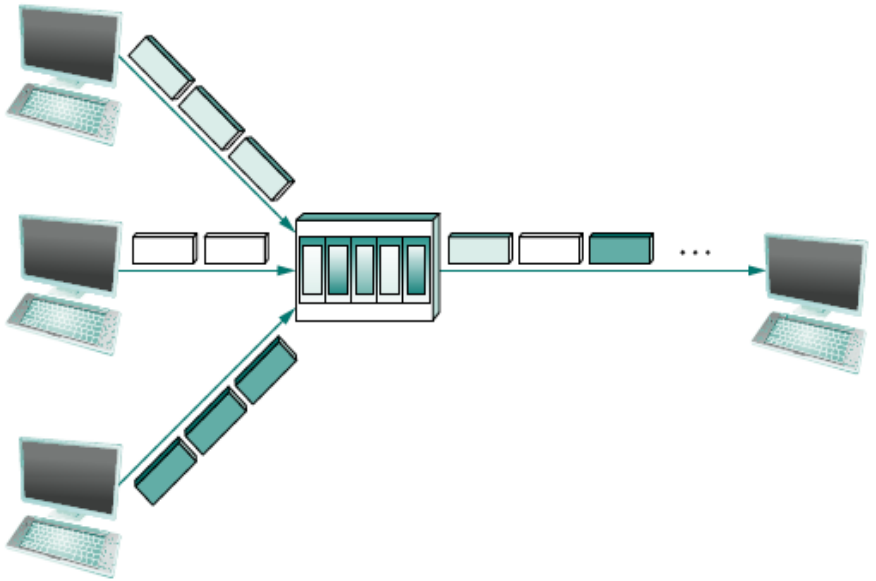
De ce este nevoie de pachete?

Legaturile unei retele pot fi folosite simultan de transmisii paralele intre mai multe perechi de noduri

Multiplexare – se aloca transmisiilor

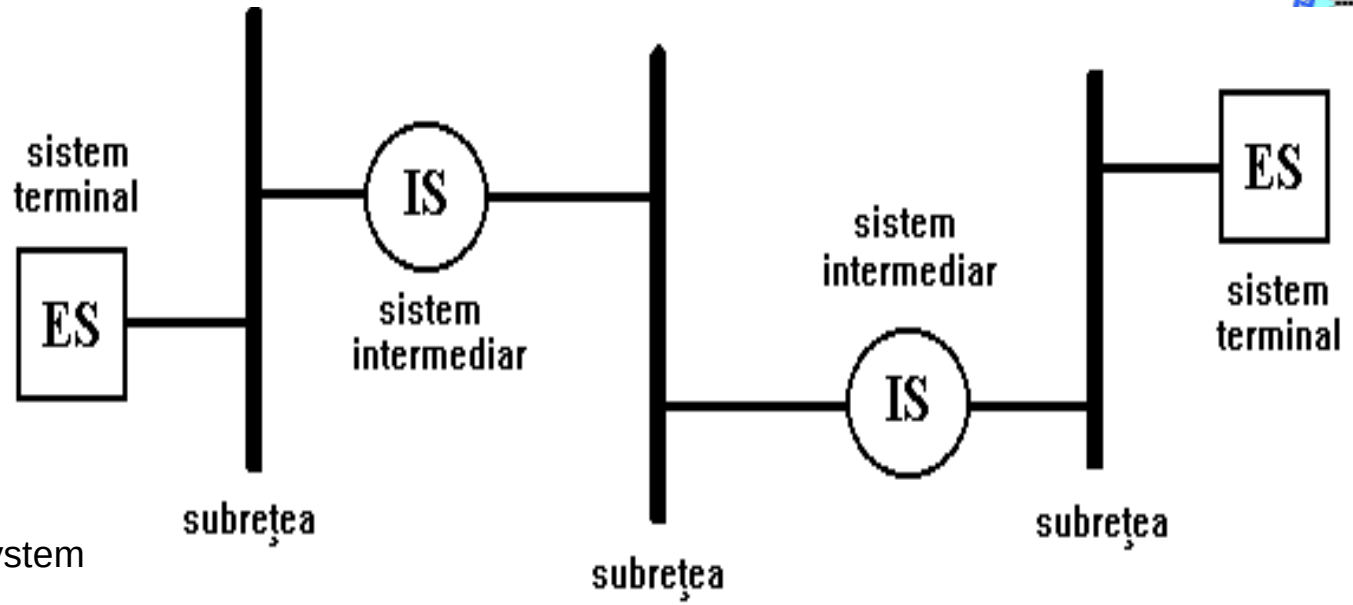
- sloturi de timp - STDM – Synchronous Time Division Multiplexing
- subcanale de frecvente diferite – FDM – Frequency Division Multiplexing

- multiplexare statistica – STMD – sloturile sunt alocate la cerere; daca o singura pereche de noduri are de transmis, nu asteapta ptr. slot;
- pentru a evita acapararea legaturii, transmisia se face in pachete cu dimensiune limitata

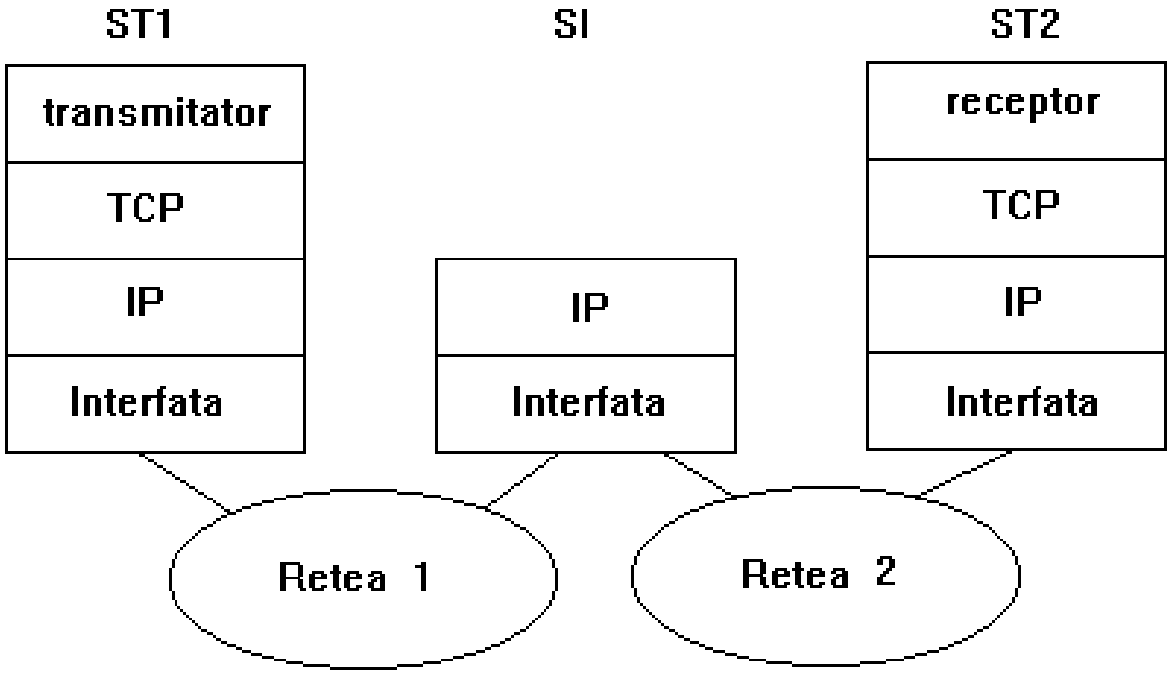


Modelul Internetului

- ES – End system
- IS – Intermediate system



Nivelele străbătute de pachete



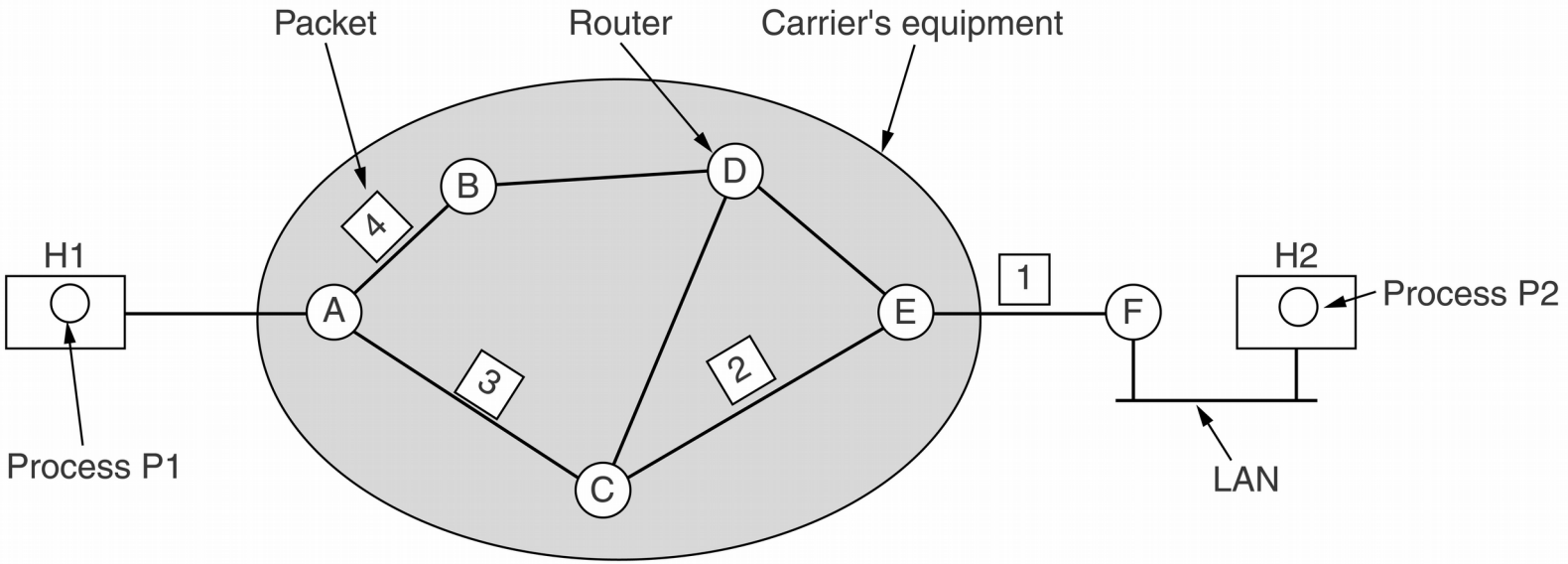
Funcţiile nivelului reţea

- dirijarea pachetelor
- adresarea

Aspecte principale

- servicii
  - ne-orientate pe conexiune
  - orientate pe conexiune
- organizarea internă corespunde tipurilor de servicii
  - datagrame
  - circuite virtuale

Organizarea internă - datagrame



A's table

Folosita de pachetele 1, 2 si 3

C's table

A	A
B	A
C	–
D	D
E	E
F	E

E's table

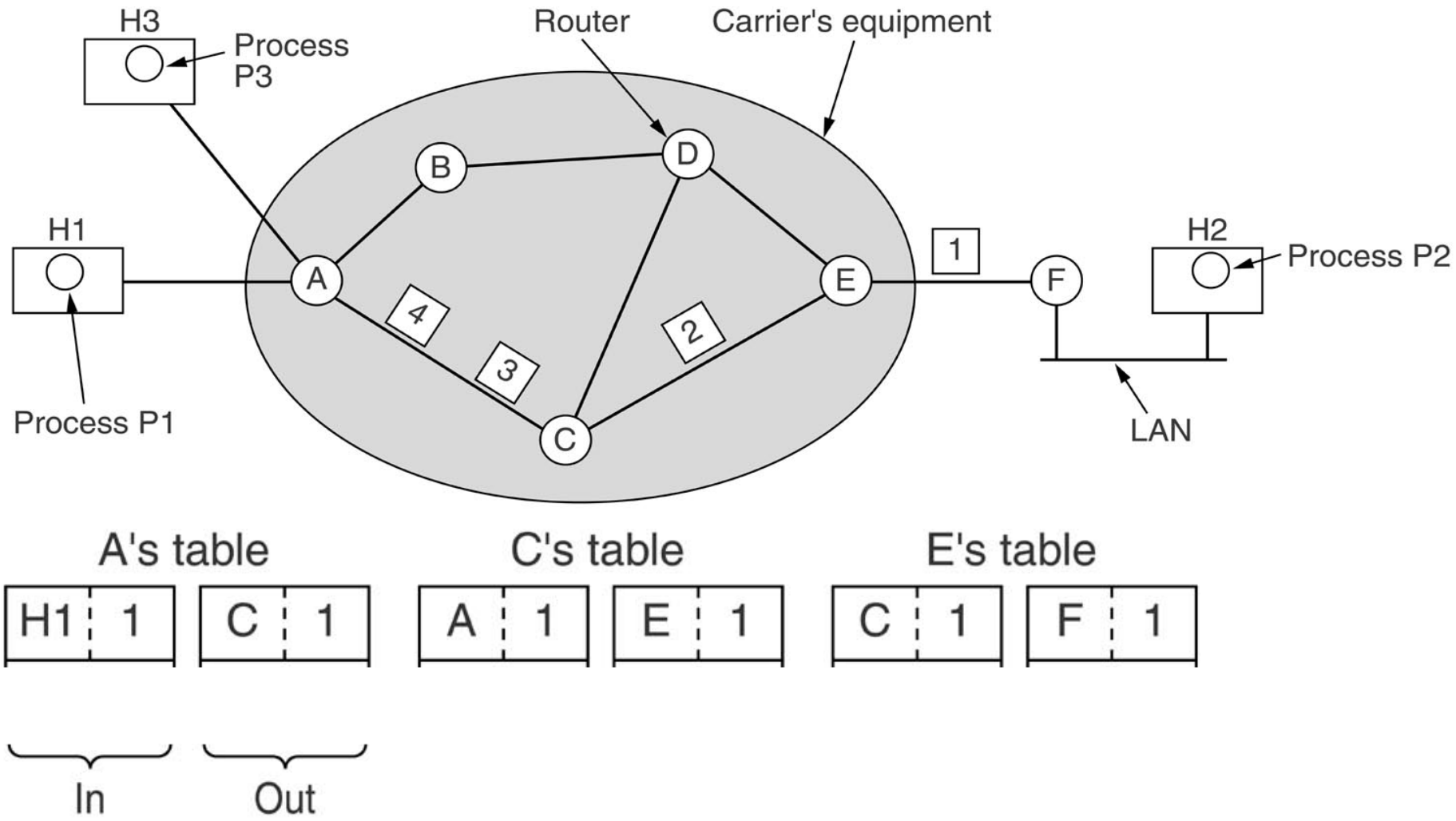
A	C
B	D
C	C
D	D
E	–
F	F

Folosita de pachetul 4

Caracteristici datagrame

- Pachetul contine toate informatiile necesare ruterelor pentru “pasarea” lui catre destinatie (inclusiv adresele sursa si destinatie)
- Organizare fara conexiune
  - Un calculator gazda (*host*) poate trimite pachetul oricand si oriunde in retea
- Nu asigura corectitudinea
  - Transmitatorul nu poate sti daca pachetul este livrat sau daca destinatarul mai este conectat
- Nu pastreaza ordinea pachetelor
  - Pachetele sunt dirijate independent unele de altele
- Robusta
  - La defectarea unei legaturi se gasesc rute alternative
- Utilizare larga in Internet

Organizarea internă – circuit virtual



Nodul A re-numeroteaza circuitul virtual

Caracteristici circuit virtual

- Organizare bazata pe conexiune
  - Transferul incepe dupa stabilirea conexiunii
  - Pachetul contine id-ul conexiunii
  - Se pot aloca resurse la stabilirea conexiunii (buffer pentru pachete)
- Transmitatorul stie
  - ca exista o conexiune
  - ca receptorul este pregatit sa primeasca pachete
- Se pastreaza ordinea pachetelor
- Se poate controla fluxul

- Folosit in
  - MPLS (MultiProtocol Label Switching)
  - retele virtuale private (VPN – Virtual Private Network)

Protocolul IP

- Are
  - o schema de adresare care permite identificarea oricarui calculator din Internet
  - un model - datagrama - pentru transmiterea datelor de la un nod gazda la altul
    - datagrama = pachet
- Modelul de serviciu – best effort
  - rețeaua “face toate eforturile” sa livreze pachetele la destinație
  - nu face nici o încercare să corecteze erorile

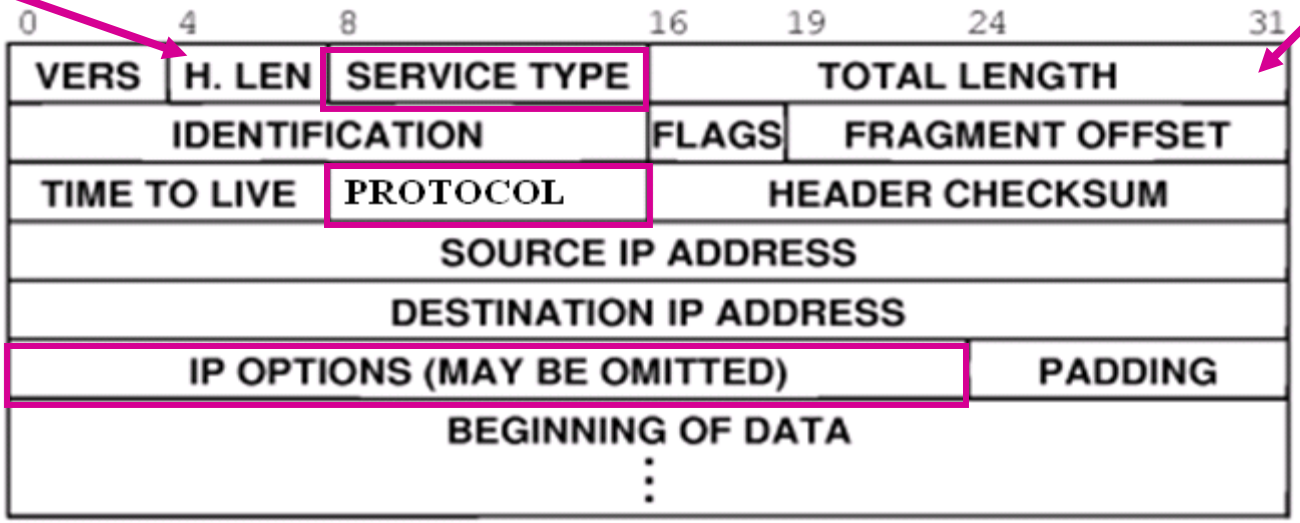
Protocol IPv4 – formatul pachetului

VERS = IPv4, IPv6….
SERVICE TYPE = precedence (3), delay, throughput, reliability, cost
PROTOCOL = (TCP, UDP, etc.)

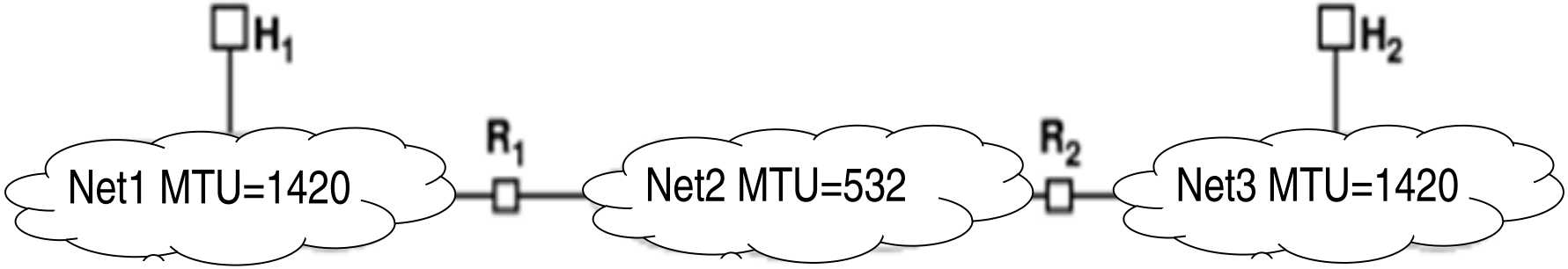
OPTIONS:
- Security
- Strict source routing
- Loose source routing
- Record route
- Timestamp

numar cuvinte de 32 biti

max 65535 octeti

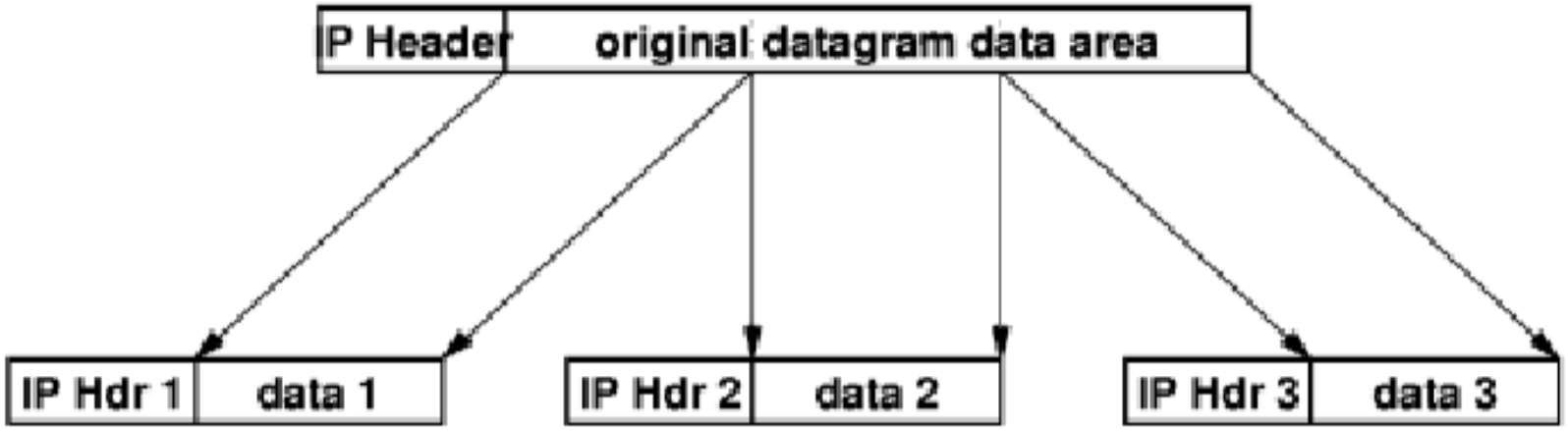


MTU – Maximum Transmission Unit



MTU – Maximum transmission unit

Fragmentarea se face la R_1



Reasamblarea se face la H_2 – de ce?

Protocol IPv4 – formatul pachetului

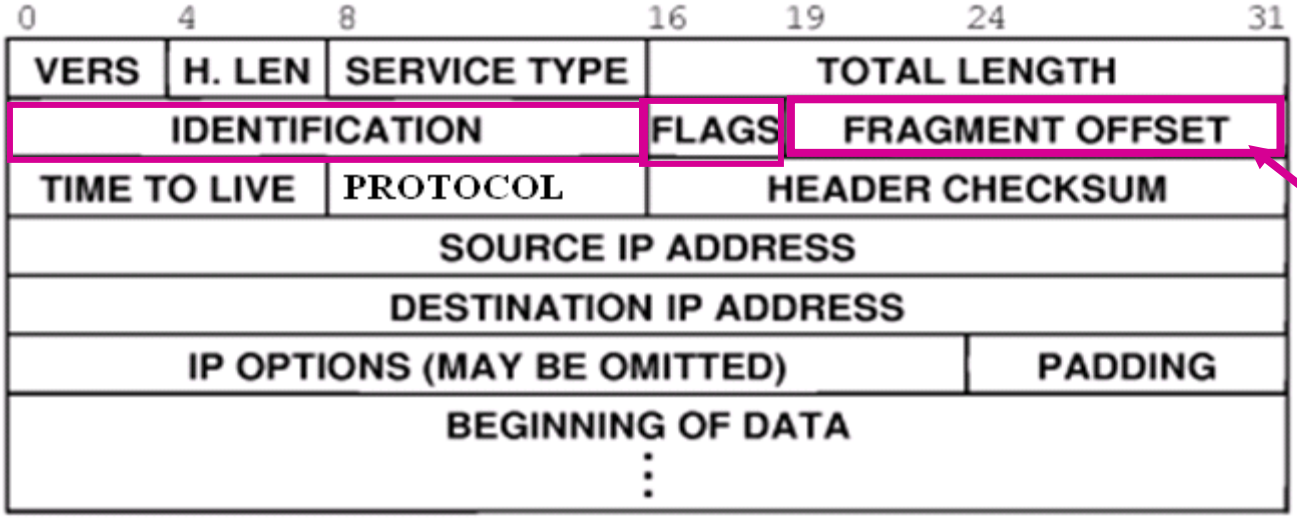
IDENTIFICATION Id datagrama de care aparţine fragmentul

FRAGMENT OFFSET pozitia fragmentului in pachet

FLAGS

DF = Don’t Fragment

MF = More Fragments



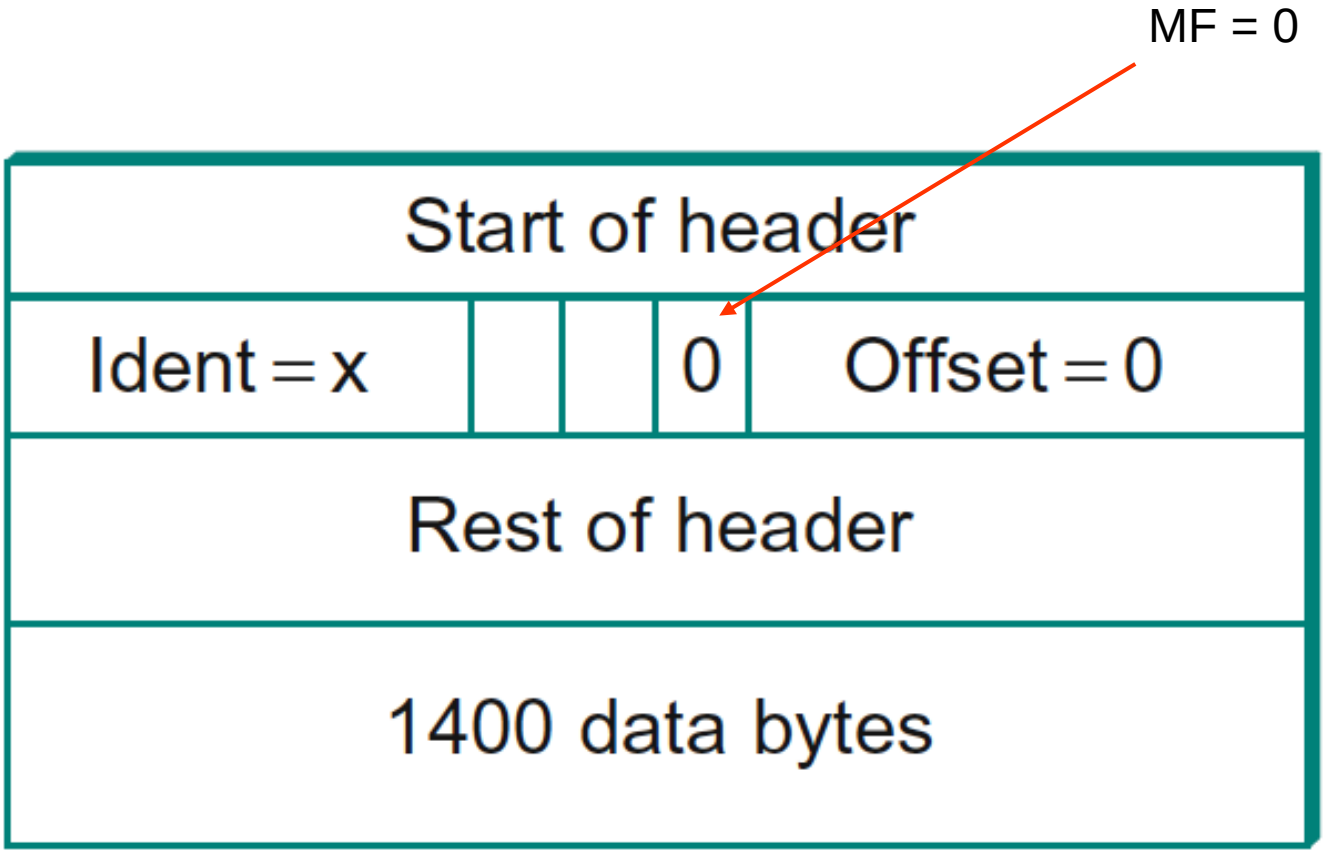
max 8192 fragmente a 8 octeti

Campurile de antet pentru fragmentare (1)

a) pachet transmis ne-fragmentat prin Net1 cu MTU = 1420

are: 1400 octeti de date

20 octeti de antet IP



Campurile de antet pentru fragmentare (2)

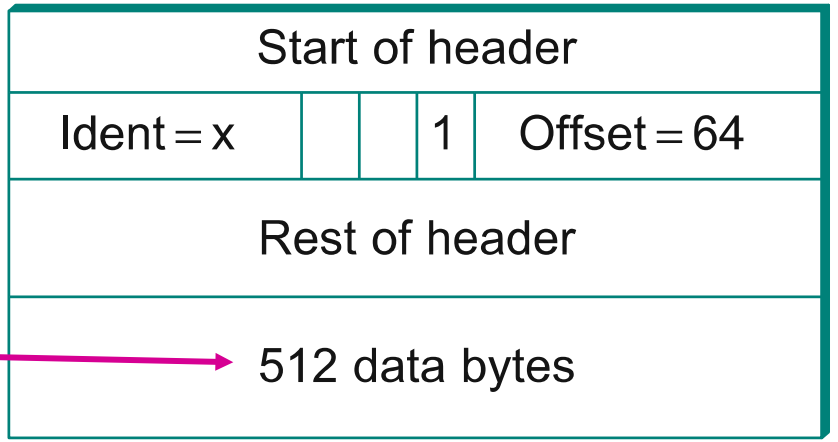
Net2 are MTU = 532

Ruterul R1 segmenteaza pachetul in 3 fragmente (fig b1-b3)

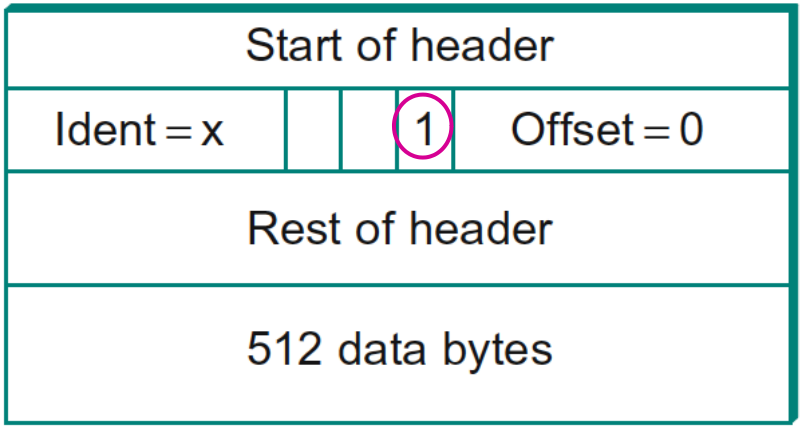
- lungimea fiecarui segment este multiplu de 8 octeti
- offset-ul numara grupuri de cate 8 octeti
- Offset = 64 = 512 / 8

unde e restul pana la 532 octeti?

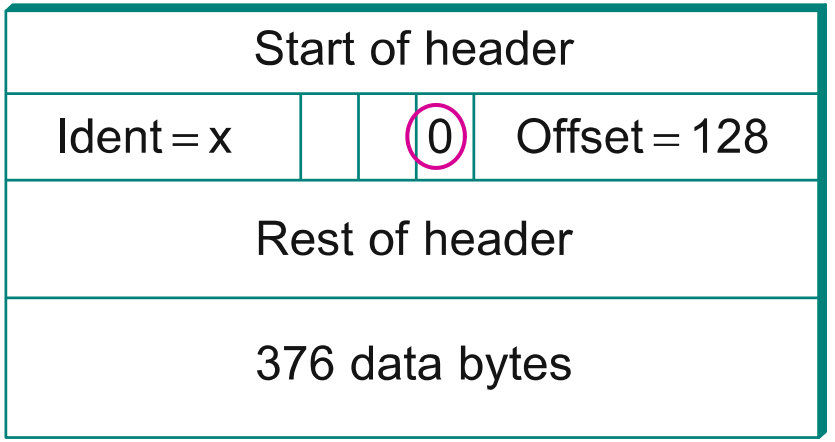
b2



b1

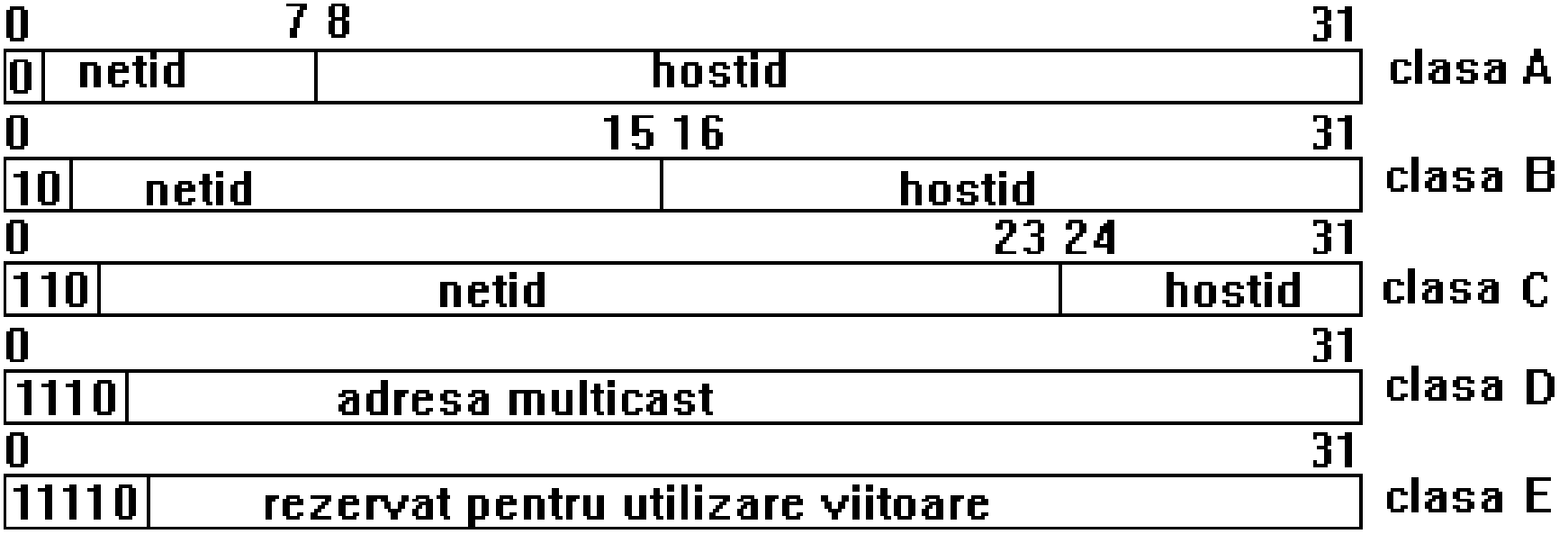


b3



Clase de adrese IP

Fiecare nod (gazda/ruter) are o adresa IP asociata cu interfata lui de retea
Un nod cu mai multe interfete are mai multe adrese IP (ex. rutere)



Clasa de adrese	Biţi în prefix	Număr maxim de reţele	Biţi în sufix	Număr maxim de gazde per reţea
A	7	128	24	167777216
B	14	16384	16	65536
C	21	2097152	8	256

Câteva adrese speciale

Prefix (rețea)	Suffix (gazdă)	Semnificație	Scop
toţi 0	toţi 0	acest calculator	Folosită cand nodul inca nu are o adresa
network	toţi 0	network	Identifică reţeaua
network	toţi 1	broadcast	broadcast în reţeaua specificată
toţi 1	toţi 1	broadcast	broadcast în reţeaua locală
127	orice	loopback	testare

Notații pentru adrese

binară 11000010 00011000 00010001 00000100

zecimală 194.24.17.4

Tabele de dirijare

Orice pachet conține o adresa IP a destinatarului, cu două părți

<adresa_retea, adresa_nod>

Un pachet este transmis de la sursă la destinație trecând prin noduri intermediare (rutere), fiecare legând între ele cel puțin două rețele

Rol ruter – primește un pachet și

- îl livrează gazdei de destinație (dacă este in aceeași rețea)
  - Toate nodurile care au aceeași adresa_retea sunt situate in aceeasi retea fizica și pot comunica direct prin legătura de date (transmit cadre)
- altfel, il re-transmite (*forward*) către un alt nod NextHop
  - Folosește **tabela de** dirijare (**rutare**) care are intrări de forma

<adresa_retea, NextHop>

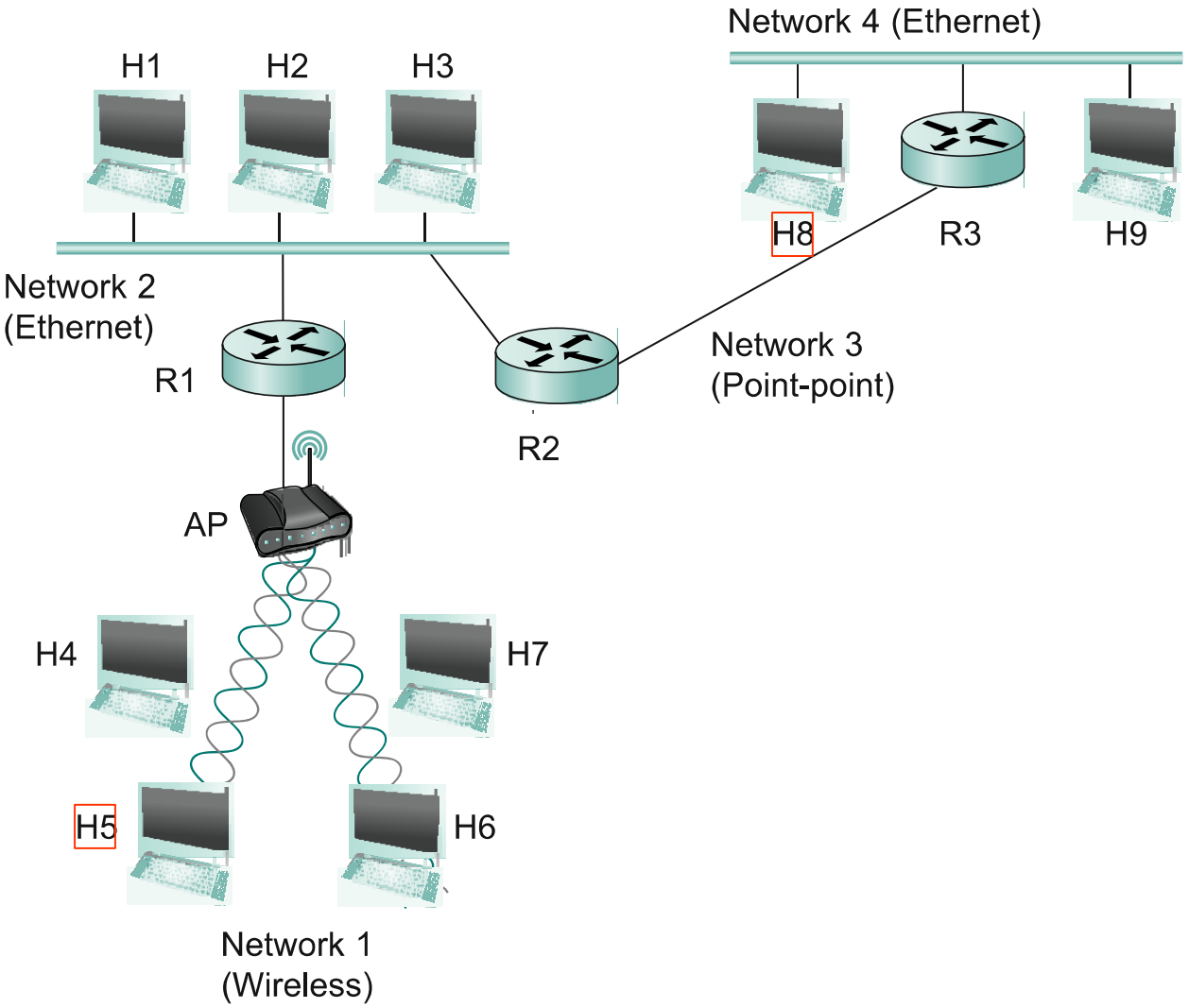
Algoritm de *forwarding* IP

Extrage <adresa_retea, adresa_gazda> destinatie din datagrama
Caută o intrare cu adresa_retea în tabela de dirijare
if adresa_retea apare in tabela de dirijare
    if adresa_retea indica o retea direct conectata **then**
        transmite datagrama direct la adresa_gazda
    else transmite datagrama urmatorului ruter (Next Hop)
else transmite datagrama unui ruter implicit

Adresarea ierarhica <adresa_retea, adresa_gazda> reduce numarul de intrari in tabela de dirijare (o intrare pentru o adresa_retea)
In practica, tabelele de rutare sunt separate pe clase de adrese
- Căutare prin: indexare (A şi B) sau hashing (C)

Exemplu

Transferul unei datagrame intre H5 si H8
H5 are legatura (printr-un punct de acces AP) la ruterul R1
Pachetul trece prin ruterele R1, R2 si R3
R3 este in aceeasi retea cu H8
si ii livrează direct datagrama



Conversie adresa IP – adresa fizică

La livrarea directă (destinatar in aceeasi retea) se foloseste adresa fizică a receptorului (ptr care se cunoaște adresa IP)
Nu exista o legatură biunivocă între adresa fizica si adresa IP
De ex.

- adresa IP are 32 biti
- adresa Ethernet are 48 biti

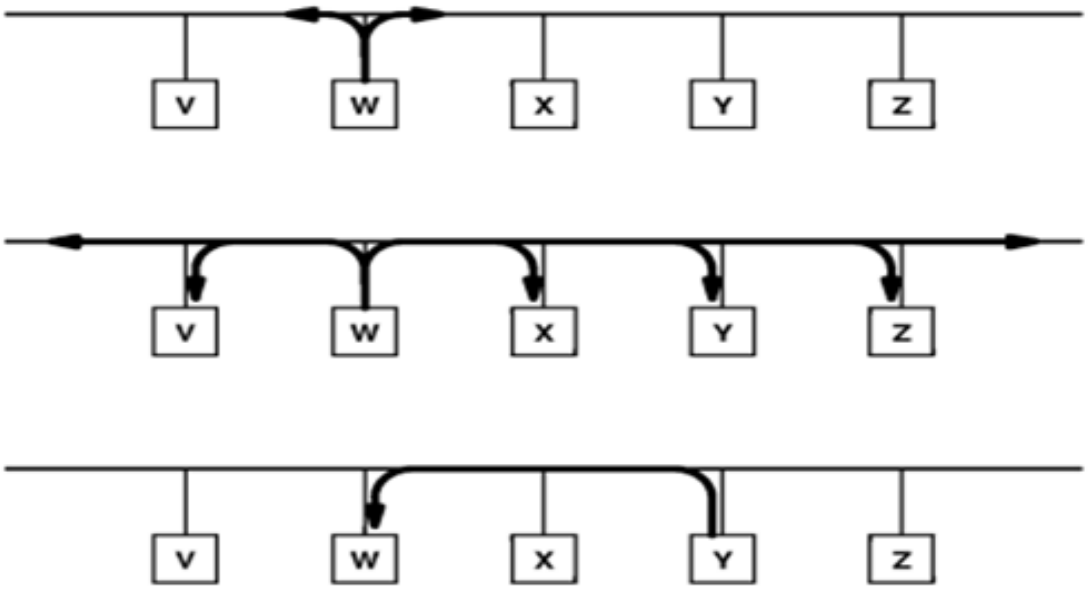
Pentru mapare se pot folosi

- tabele de corespondenţă
- formule de calcul
- schimb de mesaje
  - ex. ARP - Address Resolution Protocol
    - Face maparea intre adresa de protocol şi adresa hardware

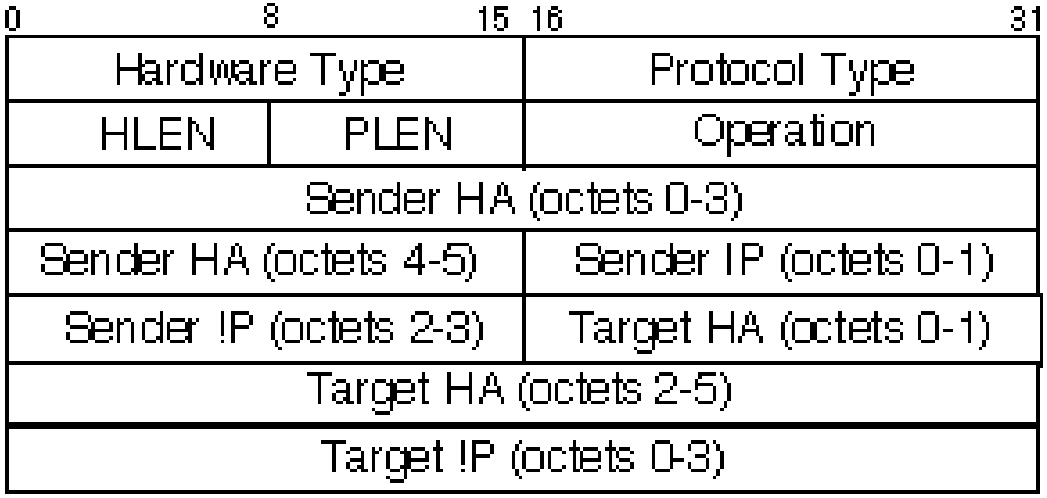
ARP - Address Resolution Protocol

Secvența de mesaje pentru a afla adresa fizică

- w difuzeaza o cerere ARP cu adresa IP cunoscută
- cererea este primită de toate gazdele din reteaua locala
- y – care are adresa IP respectivă – trimite ca raspuns adresa fizică (ex. Ethernet)



Conversie adresa IP – adresa fizică



Format mesaje ARP – conceput pentru diverse categorii de adrese

- Hardware type: 1 pentru Ethernet
- Protocol type: 0x0800 pentru IP (0000.1000.0000.0000)
- HLEN - Hardware len: 6 octeti pentru Ethernet
- PLEN - Protocol len: 4 octeti pentru IP
- Operation: 1=cerere, 2=raspuns

Subreţele

- Regula “o adresă pentru fiecare rețea fizică separată”
  - foloseste ineficient spatiul de adrese
    - o retea de clasa C cu 2 noduri consuma doar 2 din totalul de 255 de adrese de nod
    - o retea de clasa B cu peste 255 noduri ocupă peste 64000 de adrese indiferent daca le foloseste pe toate sau nu
- solutia 1 – subretele
  - de ex. o retea clasa B este impartita in mai multe subretele apropiate geografic
- solutia 2 - adrese fara clase

CIDR – Classless InterDomain Routing

Ideea: alocă spaţiul de adrese IP în blocuri de lungimi diferite

Notaţia specială pentru adresa de rețea CIDR

194.24.0.0/21 -> din cei 32 de biţi ai adresei IP

adresa_retea ocupă 21 biţi

adresa_gazda ocupă 11 biti

University	First address	Last address	How many	Written as
Cambridge	194.24.0.0	194.24.7. **255**	2048	194.24.0.0/21
Edinburgh	194.24.8.0	194.24.11.255	1024	194.24.8.0/22
(Available)	194.24.12.0	194.24.15.255	1024	194.24.12/22
Oxford	194.24.16.0	194.24.31.255	4096	194.24.16.0/20

CIDR – Exemplu

Pentru a afla adresa_retea se folosesc măști

Cambridge: Adresă **11000010 00011000 00000000 00000000**

Mască **11111111 11111111 11111000 00000000**

Edinburgh: Adresă **11000010 00011000 00001000 00000000**

Mască **11111111 11111111 11111100 00000000**

Oxford: Adresă **11000010 00011000 00010000 00000000**

Mască **11111111 11111111 11110000 00000000**

University	First address	Last address	How many	Written as
Cambridge	194.24.0.0	194.24.7. **255**	2048	194.24.0.0/21
Edinburgh	194.24.8.0	194.24.11.255	1024	194.24.8.0/22
(Available)	194.24.12.0	194.24.15.255	1024	194.24.12/22
Oxford	194.24.16.0	194.24.31.255	4096	194.24.16.0/20

CIDR – reguli de alocare a adreselor

University	First address	Last address	How many	Written as
Cambridge	194.24.0.0	194.24.7. **255**	2048	194.24.0.0/21
Edinburgh	194.24.8.0	194.24.11.255	1024	194.24.8.0/22
(Available)	194.24.12.0	194.24.15.255	1024	194.24.12/22
Oxford	194.24.16.0	194.24.31.255	4096	194.24.16.0/20

Reguli pentru a avea o mască pentru un bloc de adrese

⇒ lungimea blocului trebuie sa fie o putere a lui 2

⇒ toate adresele din bloc au aceeasi adresa_retea ⇒ adresa de inceput a blocului de adrese trebuie sa fie multiplu de dimensiunea acestuia

Ex.: zona de adrese pentru Oxford începe la o frontieră de 4096 octeţi

0 **2048** **3072** **4096**



Algoritm *forwarding*

Intrare in tabela de rutare - (adresa_retea, Masca, NextHop)
Algoritmul alege intrarea pentru care
(Adresa_IP AND Masca) = adresa_retea
Ex. Sosește pachet cu adresa_IP = 194.24.17.4
compara cu Cambridge /21 – adresa_rețea = 194.24.0.0
adresa_rețea: **11000010 00011000 00000000 00000000**
Masca: **11111111 11111111 11111000 00000000**
adresa_IP: **11000010 00011000 00010001 00000100**
Adresa_IP AND Masca:
11000010 00011000 00010000 00000000
= 194.24.16.0 ≠ 194.24.0.0 → nepotrivire

Algoritm *forwarding (2)*

Ex. Sosește pachet cu adresa_IP = 194.24.17.4
Cambridge /21 – adresa_rețea = 194.24.0.0
(Adresa_IP AND Masca) = 194.24.16.0 ⇒ nepotrivire

Edinburgh /22 - adresa_rețea = 194.24.8.0
(Adresa_IP AND Masca) = 194.24.16.0 ⇒ nepotrivire
Oxford /20 - adresa_rețea = 194.24.16.0
(Adresa_IP AND Masca) = 194.24.16.0 ⇒ **potrivire**

Dacă nu sunt alte potriviri ⇒ folosește intrarea pentru Oxford

Potriviri multiple

Prefixe de lungimi diferite

⇒ unele adrese IP se pot potrivi cu mai multe adrese_retea din tabela de dirijare

Ex.

adresa_IP	171.69.10.5	se potrivește cu
adresele de rețea	171.69.0.0/16	
	171.69.10.0/24	

Regula: se alege potrivirea “mai lungă”

Reducere dimensiune tabelă rutare

Soluție - agregarea unor adrese

Consideram adresele de retea:

C - Cambridge: 194.24.0.0/21

E - Edingurgh: 194.24.8.0/22

O - Oxford: 194.24.16.0/20

C: adresa_retea **11000010 00011000 00000000 00000000**

E: adresa_retea **11000010 00011000 00001000 00000000**

O: adresa_retea **11000010 00011000 00010000 00000000**

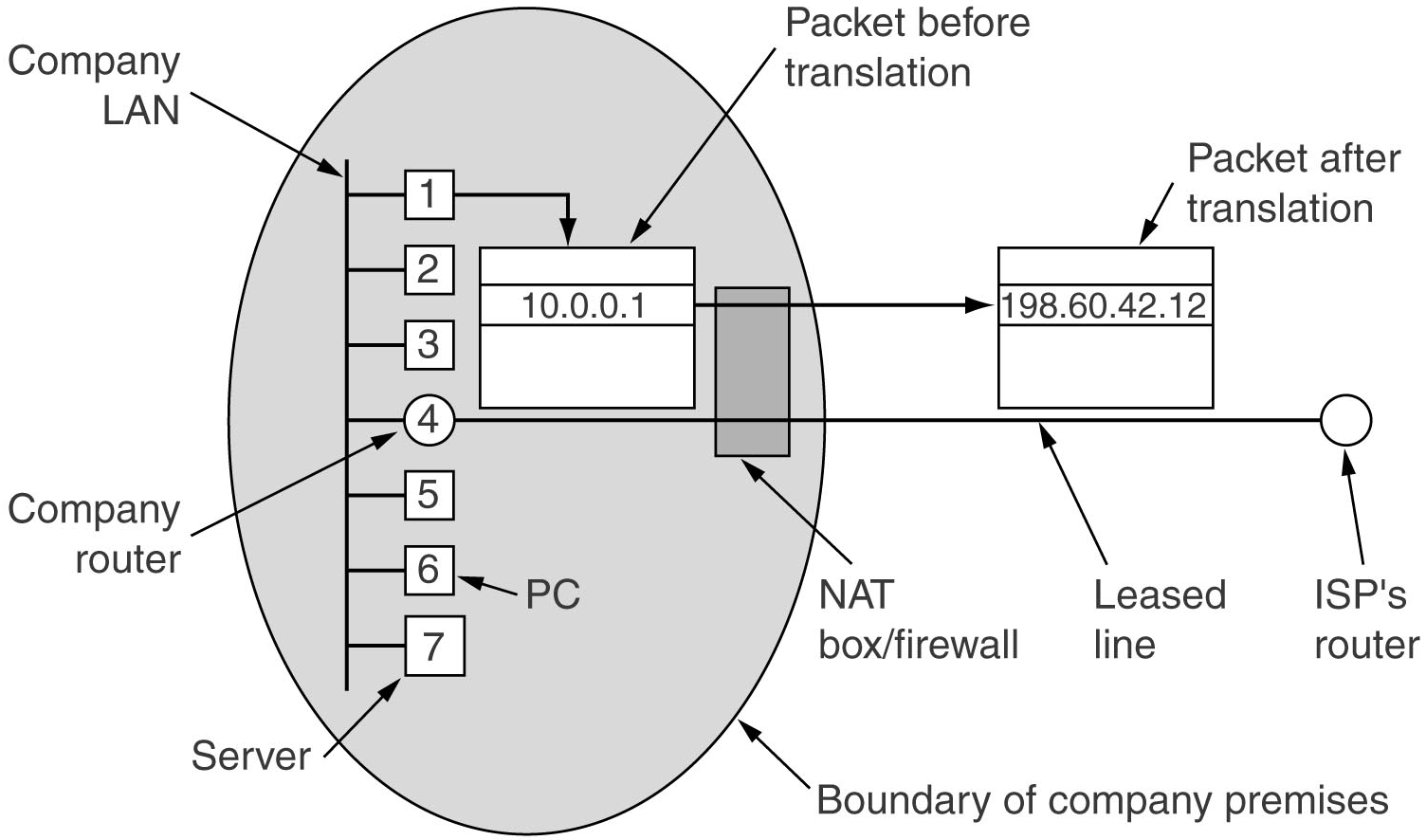
Presupunem: pentru rutare la C, E, O, nodul NewYork are in tabela de dirijare același NextHop

Tabela de dirijare poate include o singură intrare comuna pentru adresa_retea **11000010 00011000 00000000 00000000**

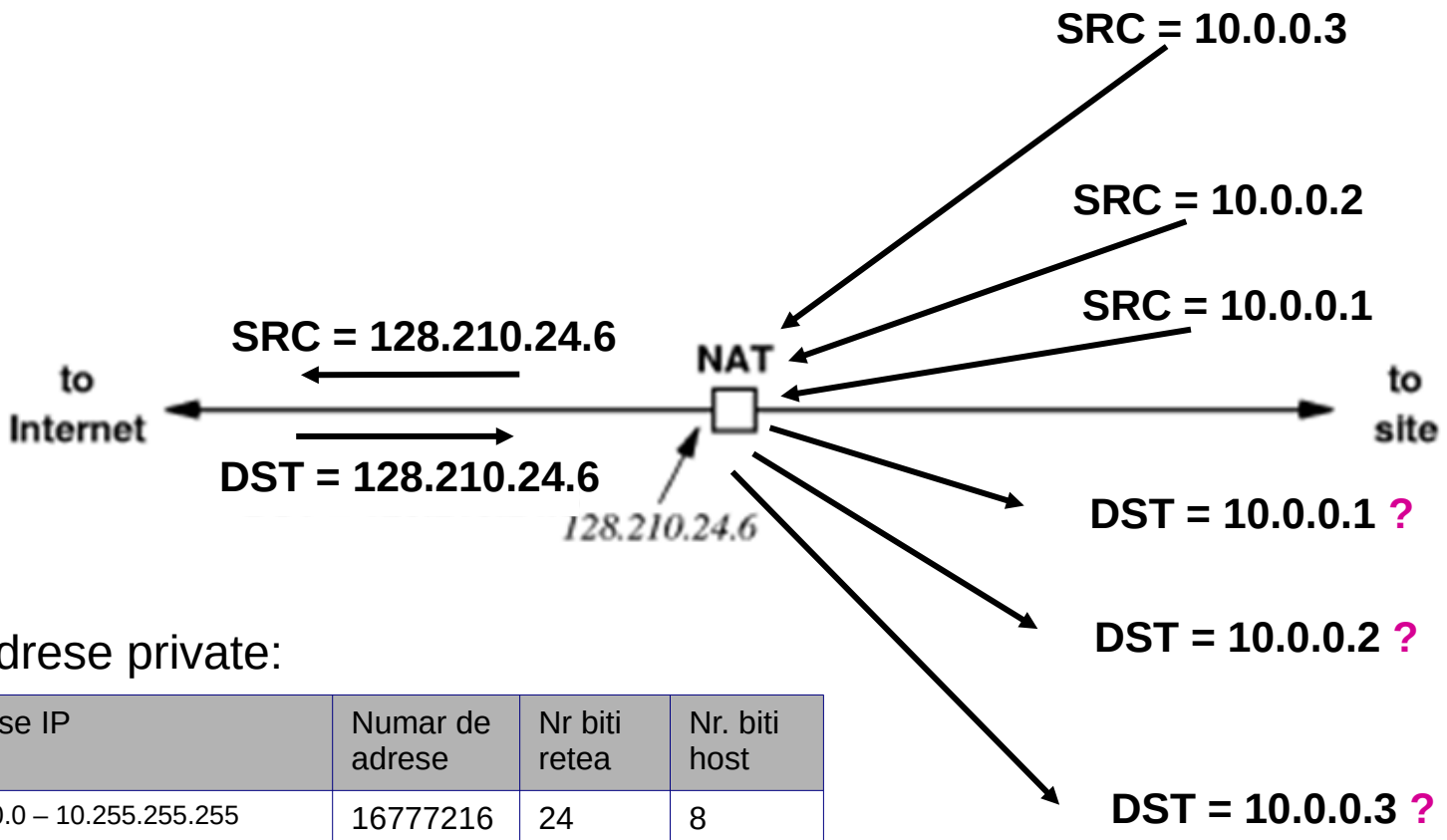
adica 194.24.0.0/19

NAT – Network Address Translation

O adresă pentru mai multe calculatoare
Foloseşte adrese locale (private sau non-rutabile)
NAT translatează între adresa privată şi o adresă globală



Translatarea adresa globală ⇒ adresa privată



Adrese private:

Adrese IP	Numar de adrese	Nr biti retea	Nr. biti host
10.0.0.0 – 10.255.255.255	16777216	24	8
172.16.0.0 – 172.31.255.255	1048576	20	12
192.168.0.0 – 192.168.255.255	65536	16	16

Principiul NAT

Foloseşte

adresa IP + număr port transmitator

tabela de translatare

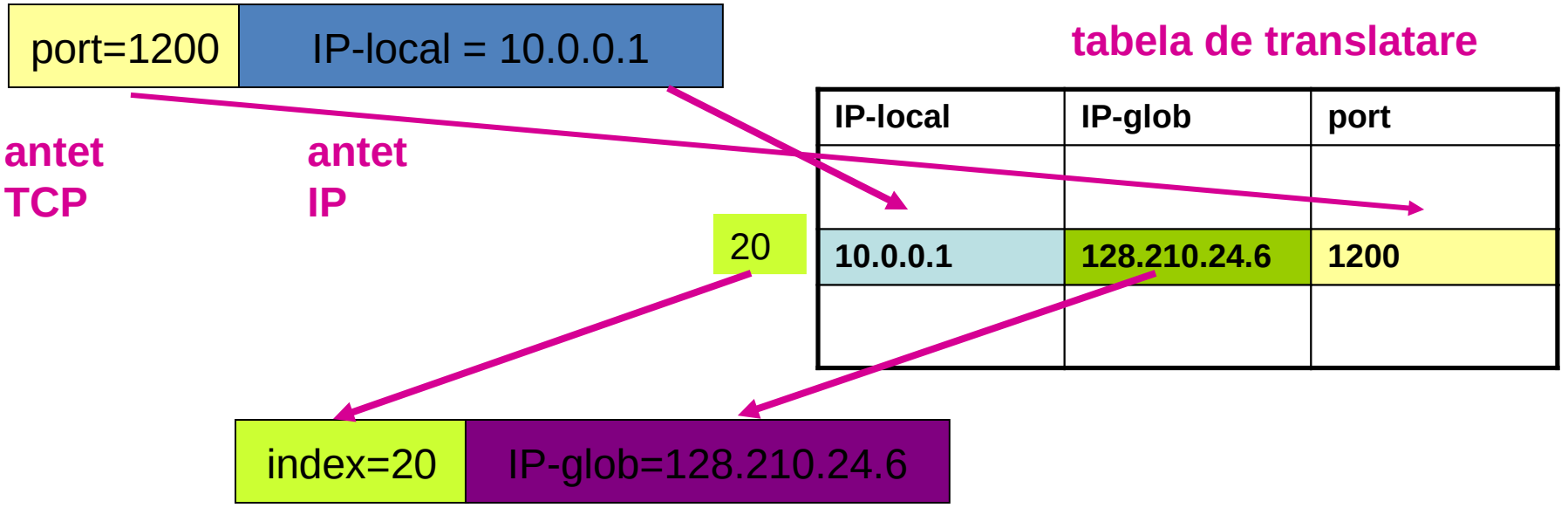
Transmisie

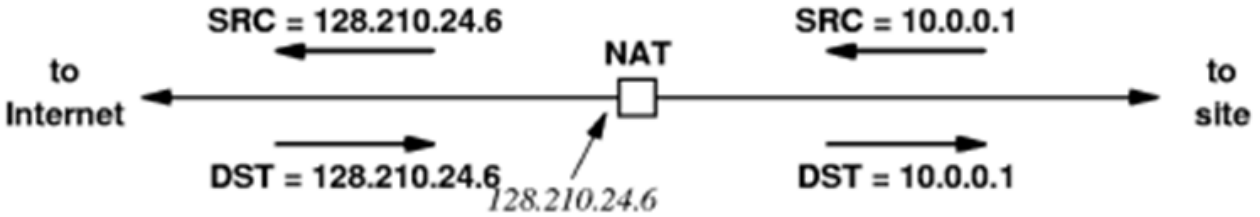
înlocuieşte adresa IP locală cu o adresă IP globală

memorează (in tabela de translatare) corespondenţa şi număr port

inlocuieşte număr port cu index în tabela translatare

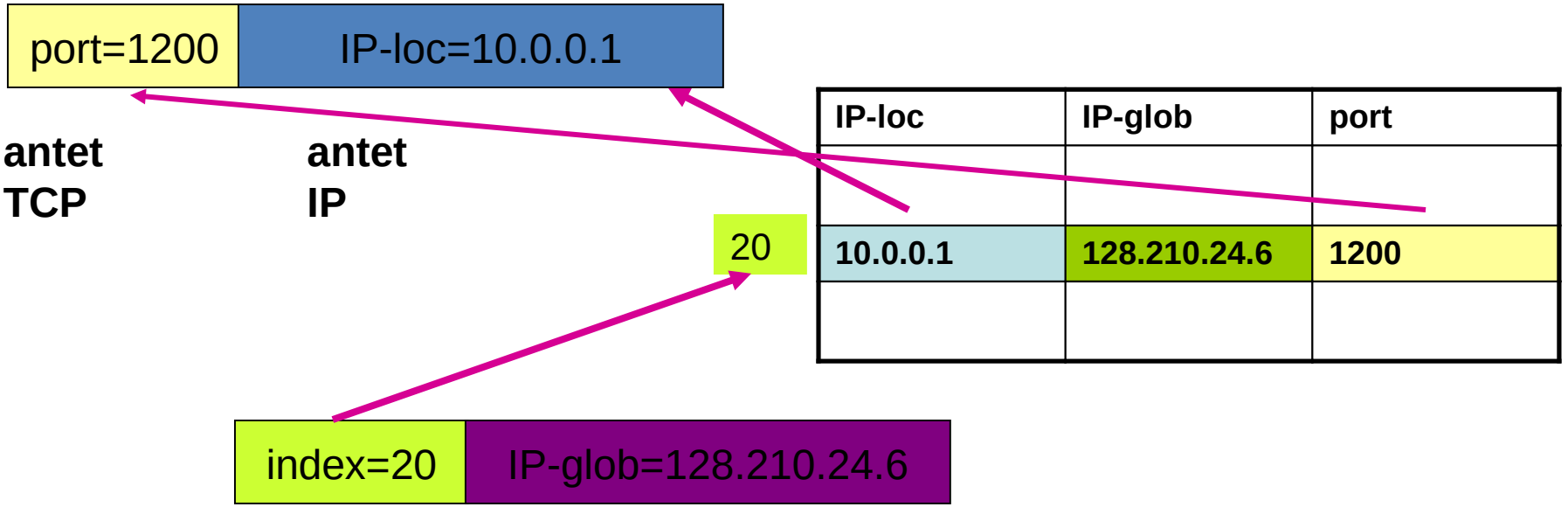
re-compune sumele de control IP şi TCP





Receptie

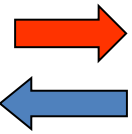
obţine număr port din pachet (= index în tabela translatare)
extrage adresa IP locală şi număr port
înlocuieşte adresa IP şi număr port din pachet
re-calculează sumele de control IP şi TCP



IP-loc	IP-glob	port
10.0.0.1	128.210.24.6	1200

ICMP- Internet Control Message Protocol

Message type	Description
Destination unreachable	Packet could not be delivered
Time exceeded	Time to live field hit 0
Parameter problem	Invalid header field
Source quench	Choke packet
Redirect	Teach a router about geography
Echo	Ask a machine if it is alive
Echo reply	Yes, I am alive
Timestamp request	Same as Echo request, but with timestamp
Timestamp reply	Same as Echo reply, but with timestamp



ICMP foloseşte IP ptr transmisie & IP foloseşte ICMP pentru raportare de erori

Test accesibilitate (**ping** trimite **ICMP** Echo şi aşteaptă un timp răspunsul)

Trasare ruta (**traceroute** trimite serie de datagrame cu valori TIME TO LIVE crescătoare şi primeşte mesaje ICMP **Time exceeded** din care extrage adresa ruterului)

ICMP- Internet Control Message Protocol

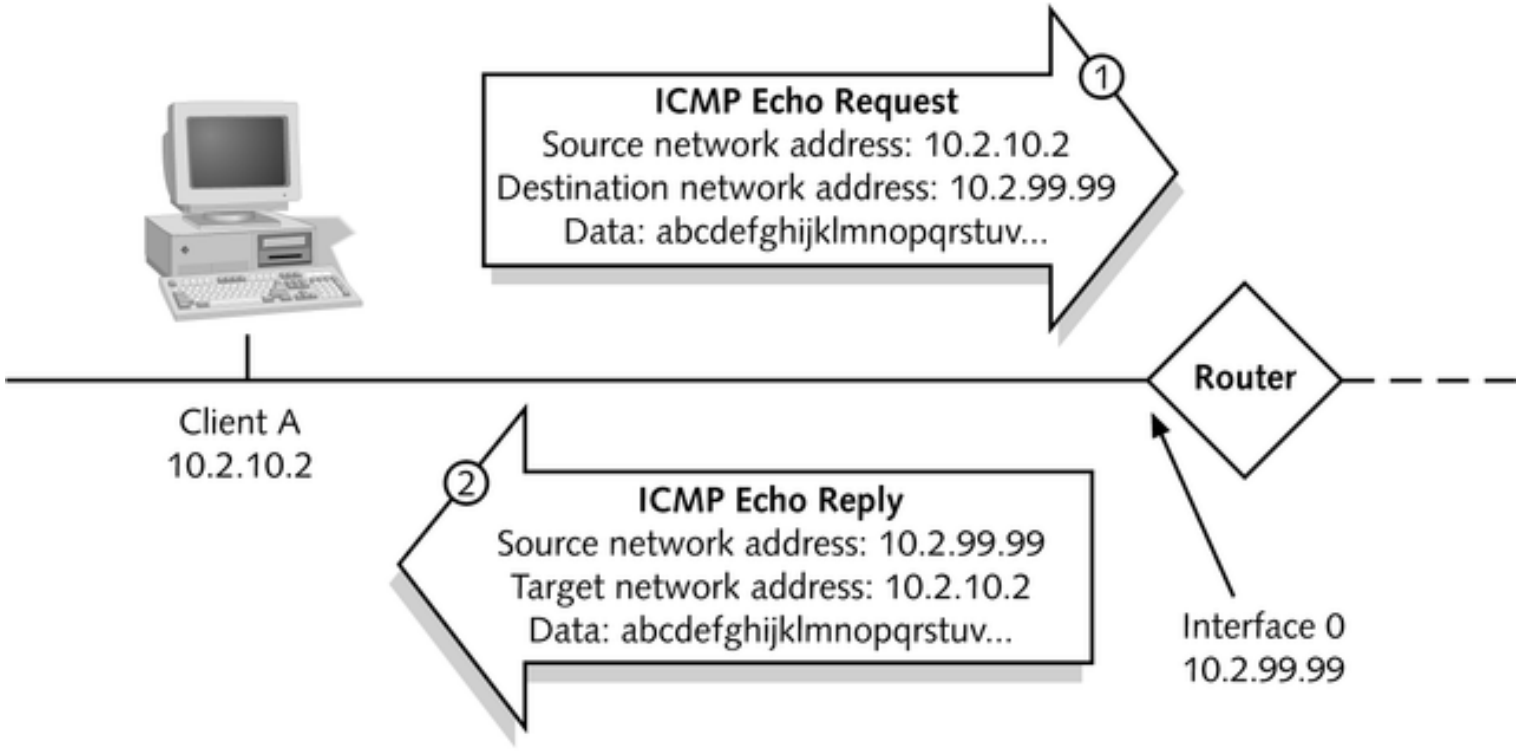


Figure 4-1 PING utility uses ICMP echo requests and replies

Functia PING foloseste mesaje de tip **ICMP echo request** si **ICMP Echo Reply**

Folosire ICMP pentru aflare path MTU

Path MTU = Maximum Transmission Unit minimă pentru o cale

- Foloseşte mesaj eroare ICMP = fragmentare necesară dar nepermisă
  - Sursa trimite probe cu DF în datagrama IP
  - Dacă datagrama > MTU => sursa primeşte eroarea ICMP

Destination Unreachable

cu Fragmentation Needed and Don't Fragment was Set

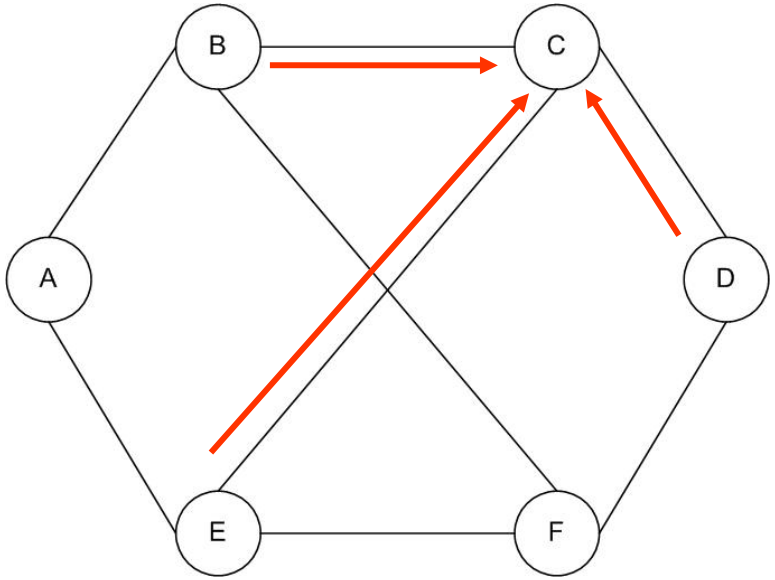
  - Sursa trimite probe mai scurte

Dirijarea - clasificare

- Fără tabele de dirijare
  - inundarea
  - hot potato
- Cu tabele de dirijare – criterii diverse
  - adaptarea la condiţiile de trafic
    - statică
    - dinamică
  - locul unde se fac calculele
    - descentralizată
    - centralizată
    - distribuită
  - criterii de dirijare
    - calea cea mai scurtă
    - întârzierea medie globală
    - folosirea eficientă a resurselor
    - echitabilitatea
  - informaţii schimbate între noduri
    - starea legăturii
    - vectorul distanţelor
  - tipul reţelei
    - uniformă
    - ierarhică

Vectorii distanţelor

Algoritm distribuit !
Fiecare nod trimite periodic vecinilor sai o lista cu distantele de la el la celelalte noduri.



Următorii vectori au fost primiţi de nodul C (lista include distanţele de la B, D, E la nodurile A, B, C, D, E, F, în această ordine):

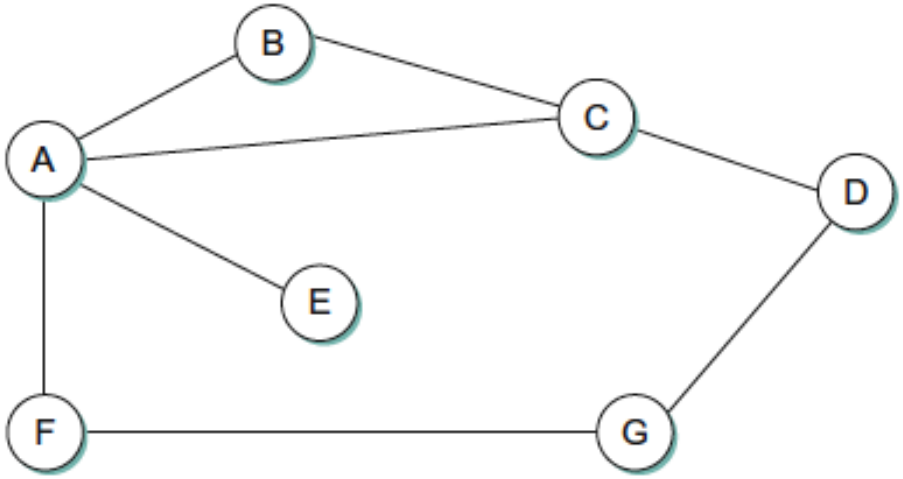
De la B: (5, 0, 8, 12, 6, 2);
De la D: (16, 12, 6, 0, 9, 10);
De la E: (7, 6, 3, 9, 0, 4).

De la / La	B	D	E
A	5	16	7
B	0	12	6
C	8	6	3
D	12	0	9
E	6	9	0
F	2	10	4

In plus, intârzierea măsurată de la C la B, D si E este 6, 3 şi 5 respectiv.

Costurile de la C / La	Prin -> B	D	E		Cost Min	Pas urmator
A	5 + 6	16 + 3	7 + 5		11	B
B	0 + 6	12 + 3	6 + 5		6	B
C	-	-	-		0	-
D	12 + 6	0 + 3	9 + 5		3	D
E	6 + 6	9 + 3	0 + 5		5	E
F	2 + 6	10 + 3	4 + 5		8	B

Problema numărării la infinit



De la	A	B	C	D	E	F	G	(cost 1 pentru orice legatura)
La E:	1	2	2	3	0	2	3	distante initiale la E
	∞	**2**	**2**	3	0	2	3	legatura A – E cade

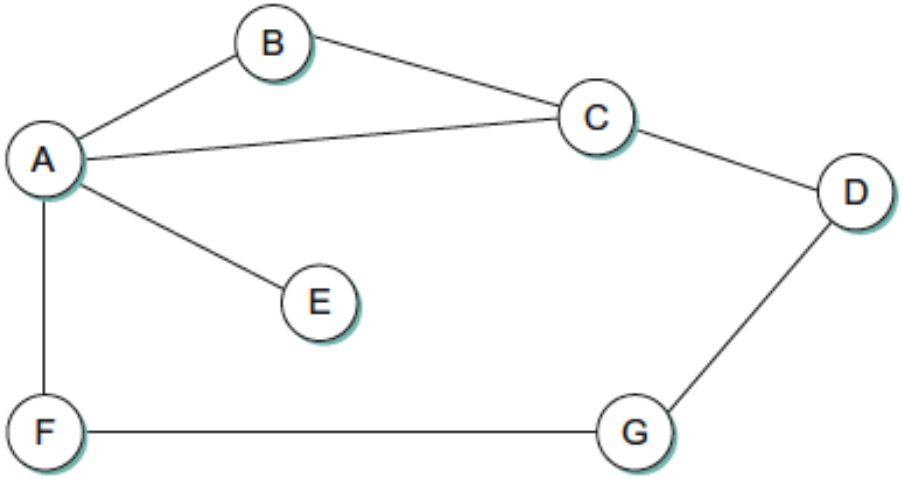
distantele catre E anuntate de: **A = ∞, B = 2, C = 2**

In functie de ordinea evenimentelor, se pot face modificarile:

	∞	**3**	**2**	3	0	2	3	B alege ruta prin C, dist=3

B anunta dist. 3 lui A

Problema numărării la infinit (2)



De la	A	B	C	D	E	F	G	
	4	**3**	**2**	3	0	2	3	A alege ruta prin B dist=4
								A anunta dist. 4 lui C
	4	**3**	**5**	3	0	2	3	C recalculeaza dist=5

distantele cresc teoretic la infinit
practic, se poate limita la un numar > diametru graf (ex. 16)

Problema numărării la infinit - solutii (3)

split horizon: noile distante nu se trimit vecinului prin care trec actualele rute

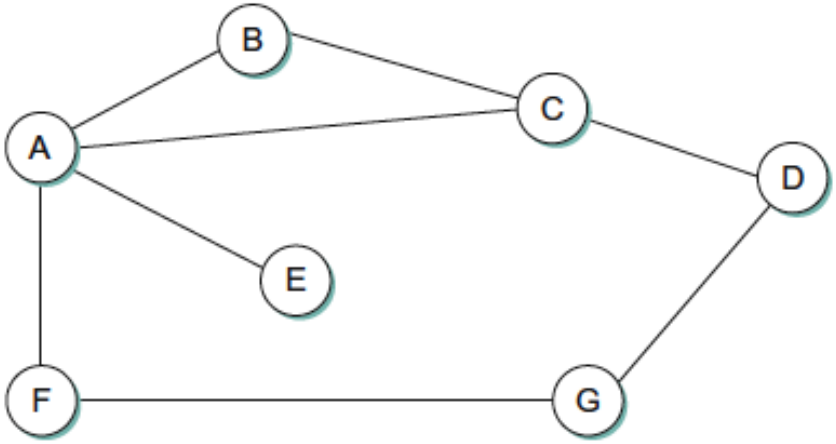
- B are ruta de distanta 2 catre E prin A
- B nu include noua distanta catre E in actualizarea trimisa lui A

split horizon with poison reverse: trimite o valoare f. mare

- B trimite distanta ∞ catre A
- A nu va mai alege o cale prin B

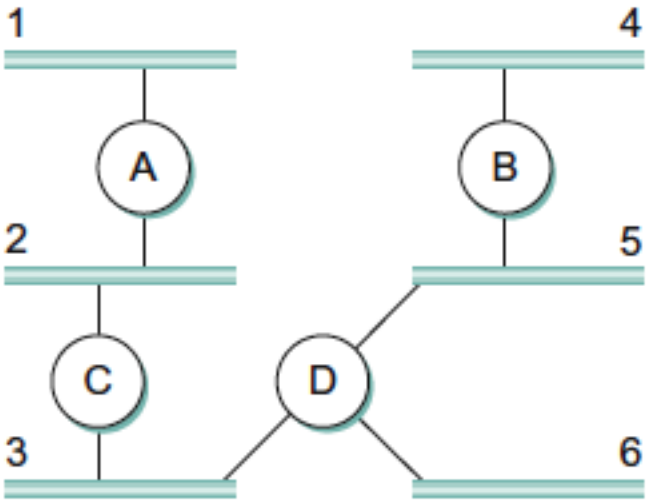
Neajuns: solutiile nu functioneaza in toate cazurile

- de ex. pentru bucle cu mai multe noduri



RIP - Routing Information Protocol

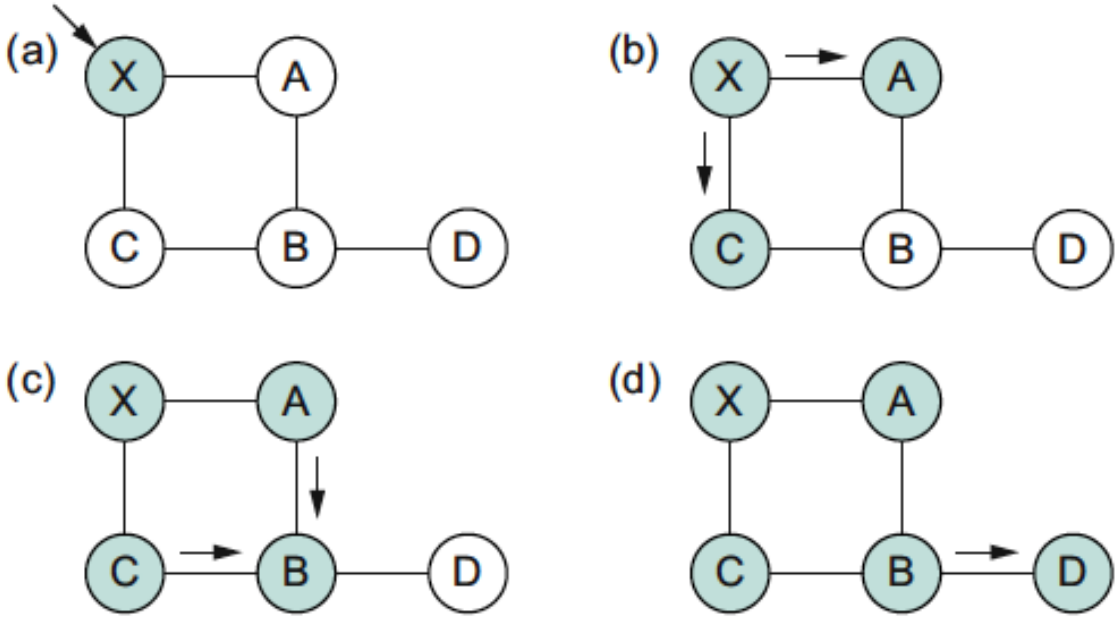
- fiecare legatura are cost 1
- foloseste distante la retele (nu la noduri)
  - ruterul C are distanta 0 la reteaua 2 si 2 la reteaua 4
- transmit vectorii distantelor la fiecare 30 secunde
- distante maxime de 15 hop-uri (16 inseamna infinit)
  - rețele de mici dimensiuni



Starea legaturii

- Presupune ca fiecare nod poate gasi legaturile cu vecinii si costul fiecarei legaturi
- Informatiile sunt diseminate prin inundare tuturor celorlalte noduri
  - LSP – Link State Packet transmis prin inundare;
  - Pachetul contine
    - Id-ul nodului care creaza pachetul
    - lista nodurilor conectate cu costul fiecarei legaturi
    - un numar de secventa
    - durata de viata a pachetului (numar)
- Cu informatiile primite, fiecare nod va calcula rutele cele mai scurte catre celelalte noduri

Transmiterea prin inundare



- ID si nr. secventa:
  - nodul are o copie a pachetului sau pachetul este vechi - ignorat;
  - pachet nou - memorat si retransmis vecinilor (mai putin vecinului de la care l-a primit)
- durata de viata:
  - numar decrementat la fiecare nod (hop) – asigura eliminare pachete vechi

Algoritmul căii celei mai scurte

Algoritmul lui Dijkstra

nnod	numărul nodurilor reţelei;
sursa	nodul sursă;
l[i][j]	costul legăturii (i,j), având valorile 0 dacă i = j; lungmax dacă i şi j nu sunt adiacente; o valoare între 0 şi lungmax în celelalte cazuri;
D[i]	costul minim al legăturii de la sursă la i;
S	mulţimea nodurilor deja selectate;
V	tabloul de dirijare; V[i] = vecinul prin care se transmit date de la nodul curent la nodul i.

```
void Dijkstra (int sursa)
{ int i, j, k;
    for (i=1; i <= nnod; i++)
    {     S[i] = 0;                  // nod neselectat
          D[i] = l[sursa][i];        // distantele minime de la sursa
          if (D[i] < lungmax)
                V[i] = i;          // initializeaza vecinii
          else
                V[i] = 0;
     }
    S[sursa] = 1;                    // selecteaza nodul sursa
    D[sursa] = 0;

for ( i=1; i < nnod; i++)
       {   gaseste nodul k neselectat cu D[k] minim;
           S[k] = 1;
           for (j=1; j <= nnod; j++)          // recalculeaza distantele
                 if ((S[j] == 0) && (D[k] + l[k][j] < D[j]))
                       { D[j] = D[k] + l[k][j];
                         V[j] = V[k];        // modifica tabela de dirijare
                       }
      }
}
```

Structura ierarhica a Internet-ului

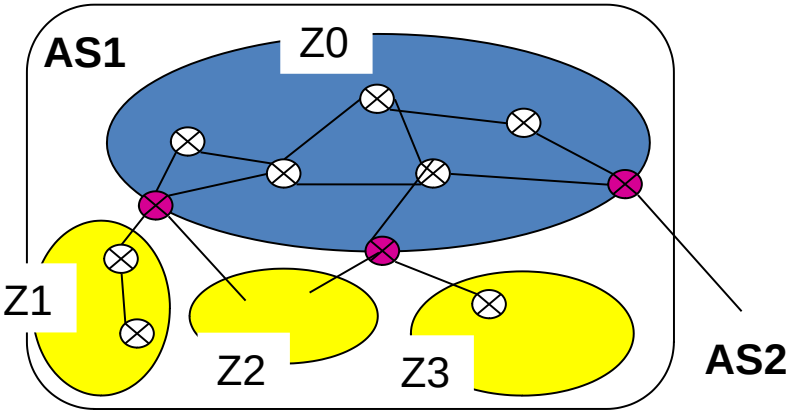
- Internet-ul este partajat in mai multe domenii numite sisteme autonome AS – Autonomous Systems
  - AS-urile sunt retele independente operate de organizatii diferite
- Intr-un AS se foloseste acelasi algoritm de rutare “intra-domeniu”
  - denumit interior gateway protocol
  - ex. OSPF – Open Shortest Path First
- Rutarea intre AS-uri (intre domenii) foloseste, de asemenea, un protocol comun
  - denumit exterior gateway protocol
  - ex. BGP – Border Gateway Protocol (bazat pe RIP)

Structura ierarhica AS

Fiecare AS este partitionat in mai multe **zone (areas)** – fiecare zonă reprezentând un grup de rețele

- zona Z0 – coloana vertebrală (backbone)
- zone "stub" Z1, Z2 … – legate la backbone
- rutele intre noduri din zone stub diferite trec prin zona Z0

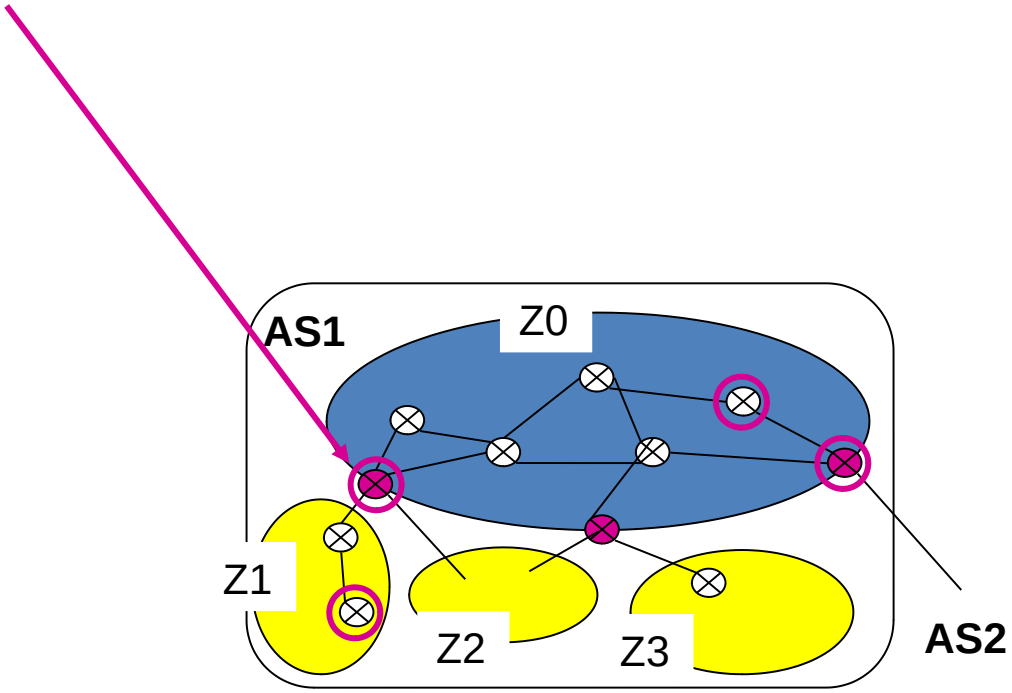
- Ierarhizarea crește scalabilitatea
  - un ruter dintr-un AS nu trebuie sa stie cum se ajunge la fiecare retea din AS, fiind suficient sa stie cum se ajunge la zona in care se afla reteaua respectiva reduce volumul tabelelor de dirijare



Structura ierarhica AS (2)

Tipuri de rutere

- **interne** unei zone
- **de coloană vertebrală** (backbone)
- **de graniţă zonală** (aparțin zonei 0 si zonelor conectate)
  - nodul incercuit face parte din zonele Z0, Z1, Z2
- **de graniţă AS**



Mesaje OSPF

Mesajele OSPF permit schimbul de informatii intre noduri
Sunt transmise in pachete IP cu 89 ca numar de protocol
Hello – stabileste si pastreaza legaturi cu vecinii
descopera nodurile (rutere) cu care este conectat direct
Link state request - Cerere stare legătură
cere info despre anumite legături de la un alt ruter
Link state update - Actualizare stare legătură
trimite info despre legaturi, ca raspuns la o cerere
Link state ack - Confirmare stare legătură
confirmă primirea unui mesaj de actualizare
Database description – trimite LSDB – LInk State Data Base
mesajele contin info despre AS sau zonă

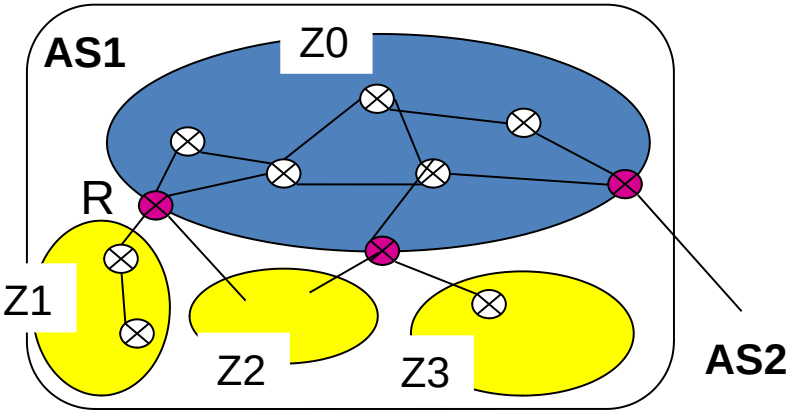
Calcul rute - Nivel 1 (zona stub)

Folosind inundarea, fiecare ruter informeaza celelalte rutere din **zona** despre legaturile sale si costurile acestora

- ex. informatiile de starea legaturilor schimbate intre noduri din Z1 nu se transmit in afara acestei zone!

Fiecare ruter (inclusiv cele de granita zonala) din **zonă** calculează separat căile cele mai scurte catre ruterele din aceeasi zona

- in final, un ruterul de graniță zonală R va cunoaste caile cele mai scurte catre oricare rețea din Z1 si Z2



Calcul rute - Nivel 2 (AS)

Ideea: calea unui pachet intre doua zone diferite are trei parti:

- de a nodul sursa la zona backbone
- traverseaza backbone
- de la backbone la reteaua de destinatie

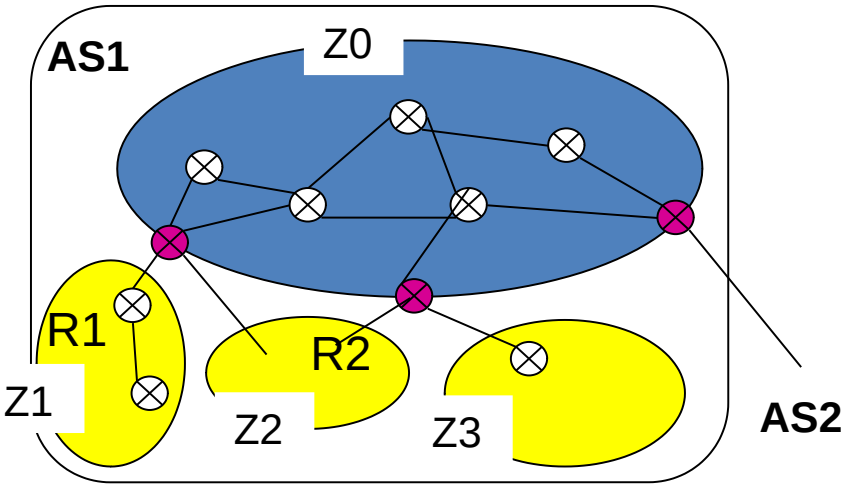
Ruterele de coloana vertebrala (backbone) primesc informatii de la ruterele de granita zonale si calculeaza cele mai bune rute la retele din orice zona

ex. R1 ofera caile cele mai scurte pentru Z1 si Z2

ruterele din Z0 calculeaza caile catre orice retea din Z1,Z2

Rezultatele sunt difuzate de la Z0 inapoi la zonele stub, care actualizeaza caile cele mai scurte la retele din alte zone.

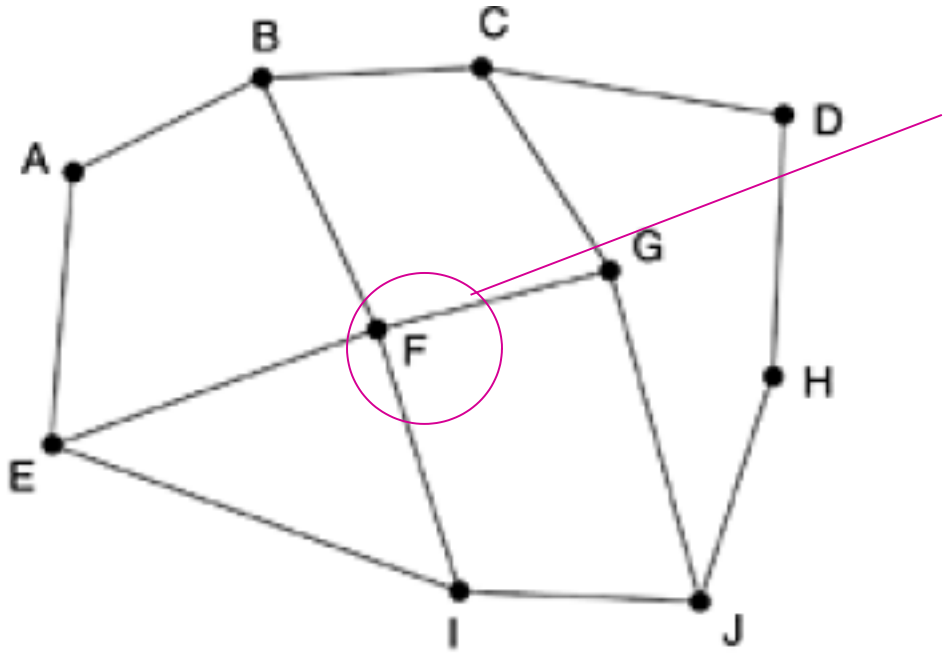
Ex. ruterele din Z2 vor sti sa aleaga intre R1 si R2 pentru rutare spre retele din alte zone



BGP – Border Gateway Protocol

Algoritmi orientaţi pe aspectele politice, de securitate, economice
Reţea = AS-uri şi conexiunile
Protocol = vectorul distanţelor
Tabelele de dirijare conţin şi rutele spre destinaţie
Comunică vecinilor căile utilizate efectiv (ocoleste numaratoarea la infinit)

Ex. pp. F foloseste calea FGCD la D



G se defecteaza
Informatiile primite de F
de la vecinii ramasi, despre D:

De la B: “Eu folosesc BCD”
De la I: “Eu folosesc IFGCD”
De la E: “Eu folosesc EFGCD”

F elimina caile care contin F
si alege calea FBCD

Dirijarea în reţele ad hoc

AODV – Ad hoc On demand Distance Vector - Determină ruta la cerere

reţea ad hoc = graf

Muchie = conexiune – nodurile pot comunica direct (radio)

Fiecare nod = ruter + gazdă

Conţine

Tabela dirijare

destinaţie,

pas următor,

distanţă

nr secv destinaţie

altele

Tabela history

identitatile cererilor precedente

Tabela reverse route

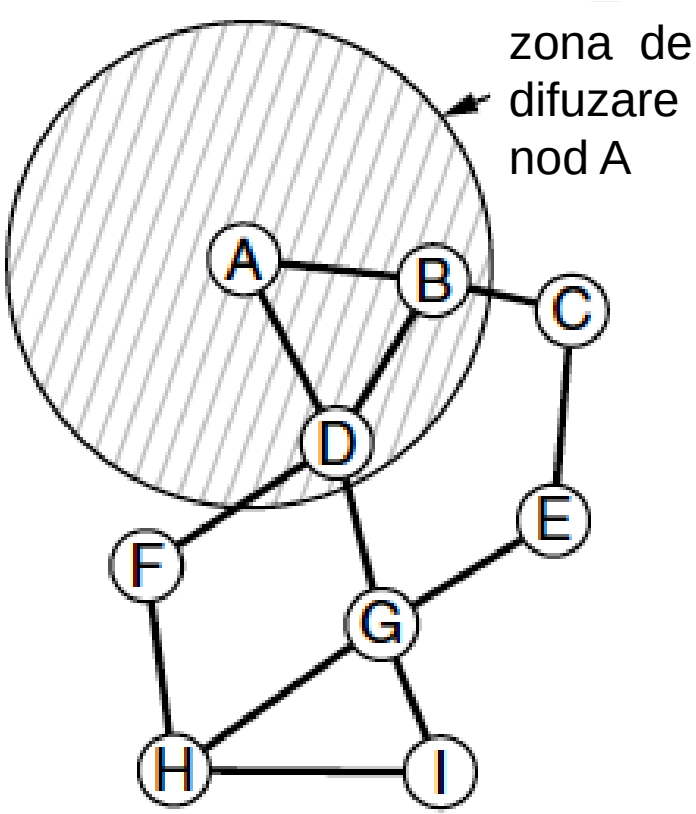
calea spre sursa unui pachet de cerere

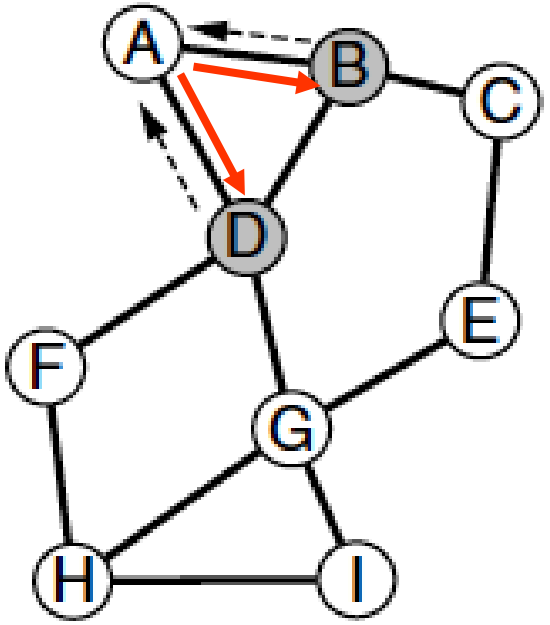
Cum functioneaza?

Exemplu: A vrea sa comunice cu I care nu e în tabela sa

trebuie să descopere ruta

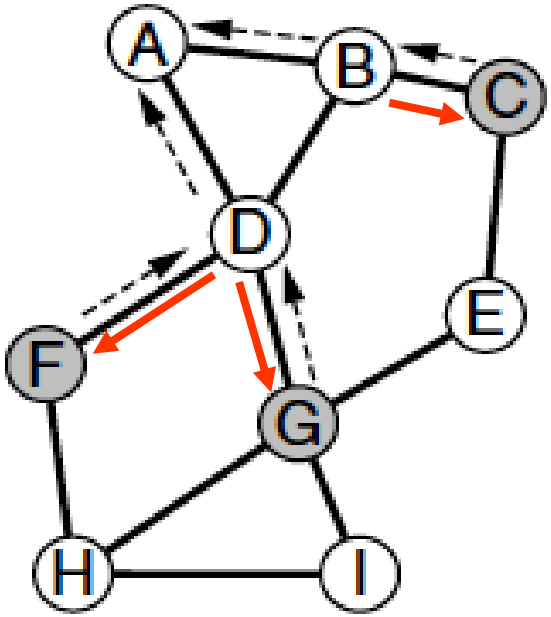
trimite (broadcast) o cerere de ruta, care ajunge la nodurile B si D

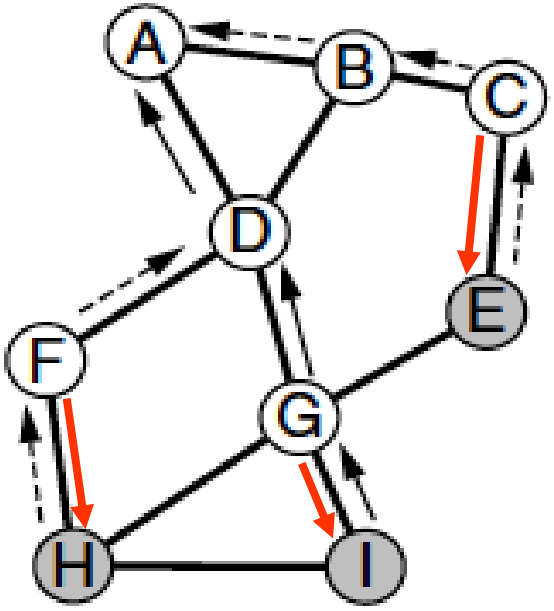




- daca B si D nu cunosc ruta spre I, cererea este re-transmisa, prin inundare, celorlalte noduri
  - mai intai C, F si G
  - apoi E, H si I
- daca cererea ajunge la I, acesta va returna un raspuns catre A
  - raspunsul parcurge calea inversa prin G si D la A (sageti negre pline)

⇒ fiecare nod trimite raspunsul vecinului de la care a primit cererea (sageti punctate)





Probleme

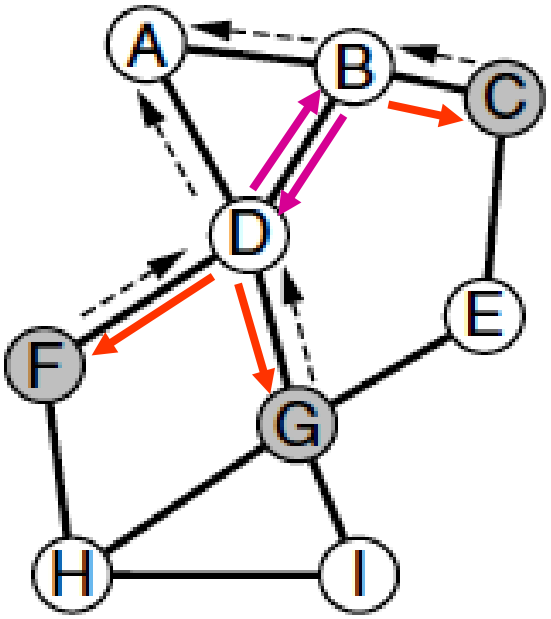
Eliminarea replicilor

- pachete transmise prin inundare
- B primeste de la D o copie a pachetului
- D primeste de la B o copie a pachetului
- pentru a descoperi si elimina copiile
  - pachetul contine un **ID al cererii**
  - nodurile pastreaza cererile primite

in **tabela history**

Lungimea caii A → I

- pachetul de cerere are un camp Hop count
- initializat de A la 0
- incrementat de fiecare nod care re-transmite pachetul
- returnat de I in raspunsul lui

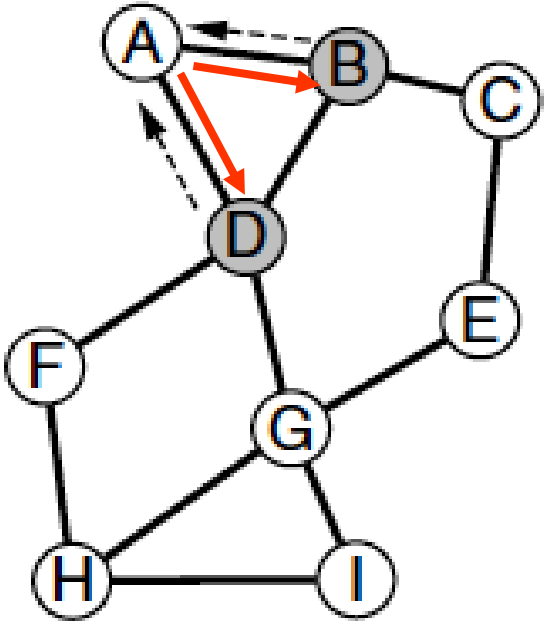


Probleme-2

Nodul A poate avea o ruta veche
- trimite o cerere pentru actualizarea ei

Se foloseşte Sequence # pentru a deosebi rutele noi de cele vechi
- inclus de A in pachet cerere
- asociat de fiecare nod rutei curente din tabela de dirijare

Exemplu
- daca nodul D are o ruta mai noua decat cea din cerere, nu va mai re-difuza cererea si va trimite raspunsul pe calea inversa



Informatii pastrate de noduri

Tabela dirijare

- destinaţie,
- pas următor,
- distanţă,
- nr secv destinaţie
- altele

Tabela history

- identitatile cererilor precedente

Tabela reverse route

- calea spre sursa unui pachet de cerere

Pachete ROUTE REQUEST

A difuzează un pachet ROUTE REQUEST
- Identificat unic prin Source address + Request ID
- Foloseşte Sequence # pentru a deosebi rutele noi de cele vechi

Prelucrarea ROUTE REQUEST în fiecare nod
- Verifica duplicat în tabela history locală (Source address + Request ID)
- Transmite ROUTE REPLY dacă găsit ruta nouă, adică
  - Dest sequence # în routing table > Dest sequence # în packet
- Altfel,
  - incrementează Hop count şi re-difuzează ROUTE REQUEST
  - memorează informaţia în reverse route table
- Source sequence # folosit pentru actualizare tabela dirijare locala

Source address	Request ID	Destination address	Source sequence #	Dest. sequence #	Hop count

Pachete ROUTE REPLY

I construieşte ROUTE REPLY şi-l trimite pe legătura inversă

- Source address, Destination address sunt copiate
- Hop count pus pe zero
- Destination sequence # luat din contorul propriu
- Lifetime = cât timp rămâne valid

Prelucrarea la alte noduri

- Actualizează tabela dirijare locală
- Transmite pe legătura inversă
- Trece prin anumite noduri – celelalte şterg intrarea în reverse route table

Source address	Destination address	Destination sequence #	Hop count	Lifetime

Intreţinerea rutelor

Actualizare: (1) la cerere; (2) la defectari

G cade → D descoperă (se folosesc mesaje Hello periodice)

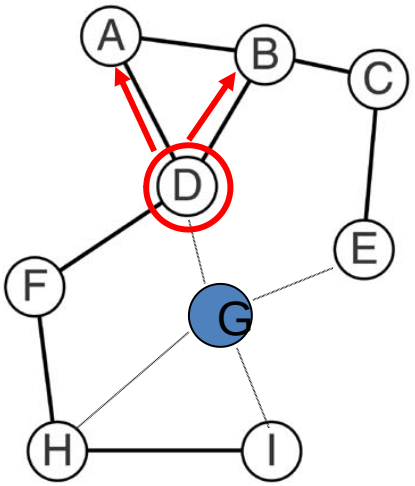
D află că G a fost utilizat pe rute către E, G şi I

D anunţă vecinii activi (active neighbors) care folosesc G, anume {A, B}

D goleşte intrările pentru E, G şi I din tabela de rutare

Dest.	Next hop	Distance	Active neighbors	Other fields
A	A	1	F, G	
B	B	1	F, G	
C	B	2	F	
E				
F	F	1	A, B	
G				
H	F	2	A, B	
I				

(a)



(b)

IPv6 - Motivaţii

Spaţiul de adrese

32 biţi = peste un milion de reţele

Dar...multe sunt Clasa C, prea mici pentru multe organizaţii

Tip servicii

Aplicaţii diferite au cerinţe diferite de livrare, siguranţă şi viteză

IPv4 are tip de serviciu dar adesea nu este implementat

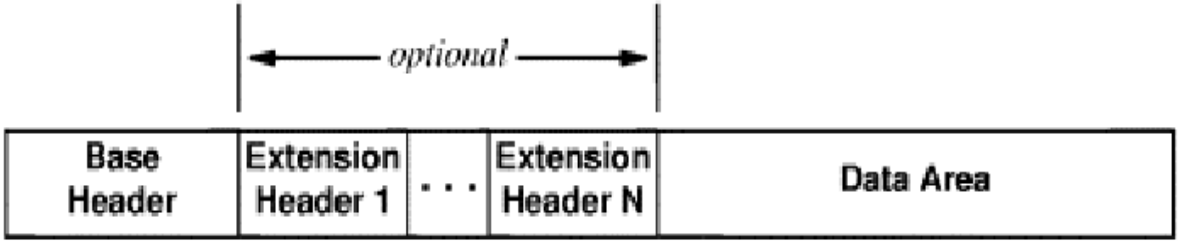
IPv6 - format Base header

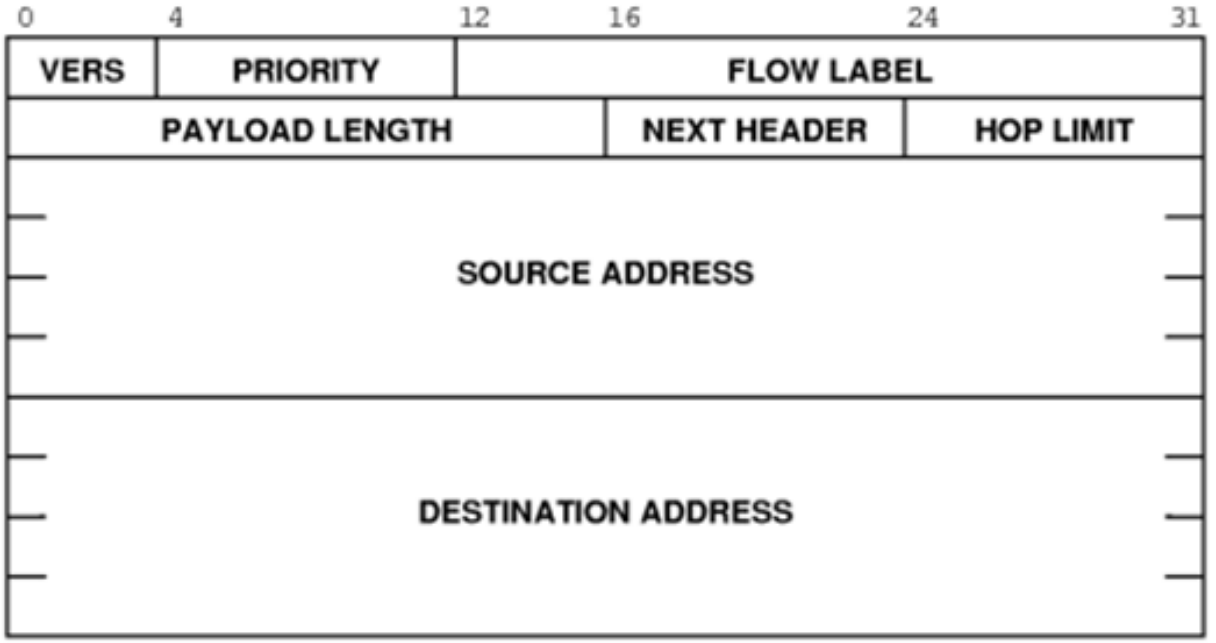
lungime fixă = 40 octeti

Priority - clasa de trafic

FLOW LABEL - asociază datagramele unui flux

Diferenţe circuit virtual
două fluxuri cu aceeaşi etichetă se diferentiaza prin adr sursă + adr dest
aceeaşi pereche sursă+dest poate avea mai multe fluxuri





Conţine mai puţine info decât antet IPv4

Restul de info în extensii

NEXT HEADER defineşte tipul datelor (ex. TCP)

NEXT HEADER defineşte tipul antetului de extensie urmator (ex. Route Header)



(a)



(b)

IPv6 – antete extensie

Hop-by-hop header – info pentru rutere – deocamdata:
suport datagrame excedând 64K (jumbograme)
specifica lungimea;
campul de lungime din antetul de baza este 0

Destination header – info aditionale pentru destinaţie nefolosit

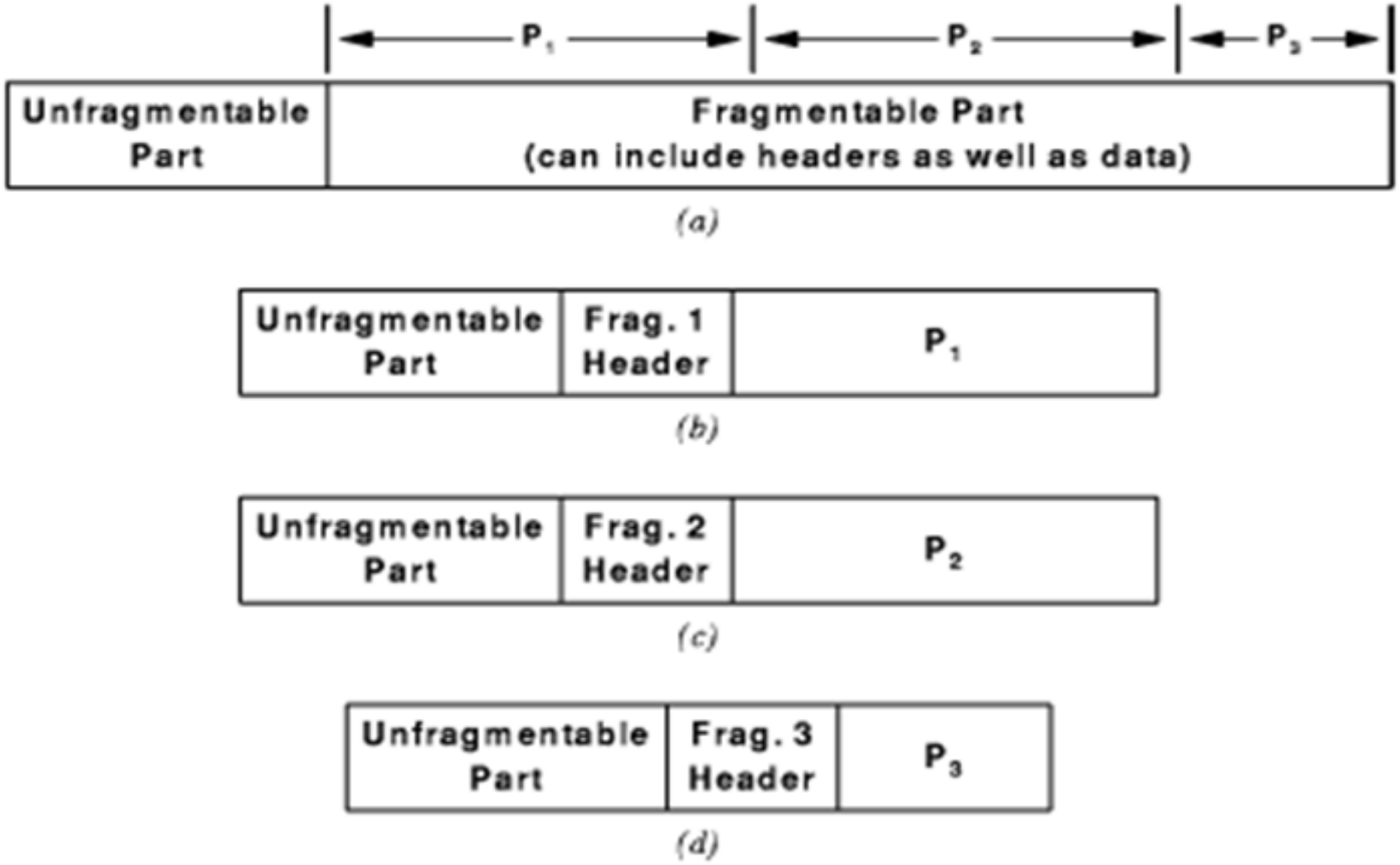
Routing – lista rutere de vizitat

Fragmentation – identificare fragmente

Authentication – verificare identitate transmiţător

Encrypted security payload – info despre conţinut criptat

Fragmentarea



Fragmentare IPv6– la sursă

Ruterele ignoră datagramele mai lungi decât MTU

Sursa

Fragmentează pachetele

Descoperă path MTU

Caracter dinamic

- calea se poate schimba

Eficienţa – antet nu are spaţiu pierdut

Flexibilitate – noi antete pentru noi caracteristici

Dezvoltare incrementală – ruterele care tratează anumite antete coexistă cu altele care le ignoră

Adrese de 128 biti

Includ prefix reţea şi suffix gazdă

Fără clase de adresă – limita prefix/suffix oriunde

Tipuri speciale de adrese:

- unicast
- multicast
- cluster – colecţie de calculatoare cu acelaşi prefix; datagrama livrată unuia din ele (permite duplicare servicii)

Notaţia adresei

16 numere
105.220.136.100.255.255.255.255.0.0.18.128.140.10.255.255

Notaţie hexazecimală
69DC:8864:FFFF:FFFF:0:1280:8C0A:FFFF

Compresie zerouri

FF0C:0:0:0:0:0:0:B1
FF0C::B1

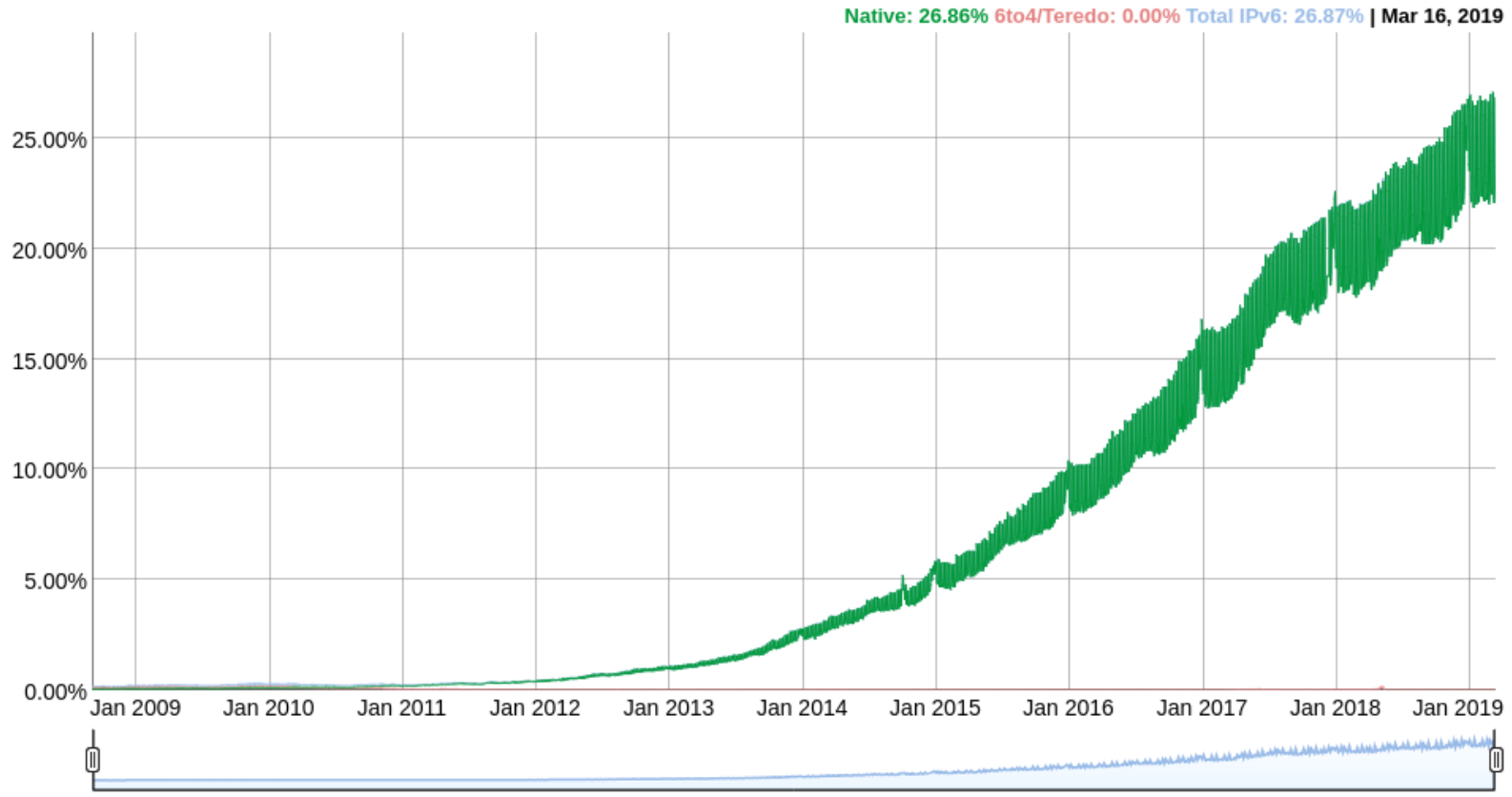
Adrese IPv6 cu 96 zerouri prefix sunt interpretate ca adrese IPv4

IPv6 adoption rate

IPv6 Adoption | Per-Country IPv6 adoption

IPv6 Adoption

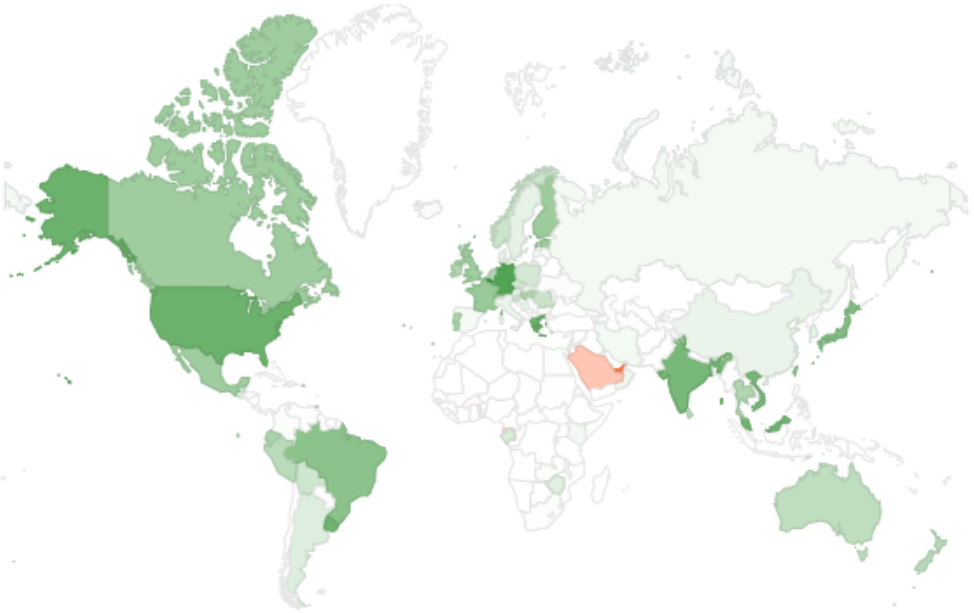
We are continuously measuring the availability of IPv6 connectivity among Google users. The graph shows the percentage of users that access Google over IPv6.



IPv6 adoption rate

IPv6 Adoption | **Per-Country IPv6 adoption**

Per-Country IPv6 adoption



World | Africa | Asia | Europe | Oceania | North America | Central America | Caribbean | South America

The chart above shows the availability of IPv6 connectivity around the world.

- Regions where IPv6 is more widely deployed (the darker the green, the greater the deployment) and users experience infrequent issues connecting to IPv6-enabled websites.
- Regions where IPv6 is more widely deployed but users still experience significant reliability or latency issues connecting to IPv6-enabled websites.
- Regions where IPv6 is not widely deployed and users experience significant reliability or latency issues connecting to IPv6-enabled websites.

Curs - Test 1

Data: 25.03.2019, ora 17:00
Durata: 30 min
Materie necesara – pana la nivelul retea (inclusiv)

Exemplu subiect:

1. (a)Descrieti protocolul cu retransmisie neselectiva „Go back n”. Care sunt avantajele și dezavantajele acestei metode? (b)Prezentati un exemplu pentru aceasta metoda de retransmitere.

2. (a)Explicati metodele de transmitere transparenta prin umplere cu biti și prin umplere cu caractere. Care sunt diferențele dintre cele doua metode? (b)Care este sirul transmis dacă se folosește metoda de umplere cu biti pentru sirul 11010111110101111100101111110110 ?

Studiu individual

A. S. Tanenbaum Reţele de calculatoare, ed 4-a, BYBLOS 2003

5.1 CERINŢELE DE PROIECTARE ALE NIVELULUI REŢEA
5.2.2 Dirijarea pe calea cea mai scurtă
5.2.3 Inundarea
5.2.4 Dirijare cu vectori distanţă
5.2.10 Dirijarea în reţele AD HOC
5.6.1 Protocolul IP
5.6.2 Adrese IP
5.6.4 Protocoale de control în Internet
5.5.5 Protocolul de dirijare folosit de porţile interioare: OSPF
5.6.5 Protocolul de dirijare pentru porţi externe: BGP
5.6.8 IPv6

Studiu individual

A. S. Tanenbaum Computer networks, 5-th ed. PEARSON 2011

5.1 NETWORK LAYER DESIGN ISSUES
5.2.2 Shortest Path Algorithm
5.2.3 Flooding
5.2.4 Distance Vector Routing
5.2.11 Routing in Ad Hoc Networks
5.6.1 The IP Version 4 Protocol
5.6.2 IP Addresses
5.6.3 IP Version 6
5.6.4 Internet Control Protocols
5.6.6 OSPF—An Interior Gateway Routing Protocol
5.6.7 BGP—The Exterior Gateway Routing Protocol